

CHAPTER 1

INTRODUCTION

1.1 Background

Financial statements are the primary documents through which an organisation reports its financial position and performance. They contain figures such as revenue, expenses, assets, liabilities, equity and profit or loss. Anyone assessing an organisation, such as a student, a small-business owner or an analyst, must read these figures and interpret them.

In practice, statements reach the reader as PDF documents or Excel workbooks. The numbers they contain are not directly usable for analysis until they are located, copied into a working format and checked. The analysis that follows requires both arithmetic and judgement.

1.2 Financial Statement Analysis Context

Financial statement analysis converts reported figures into indicators that allow performance and position to be assessed. This project concentrates on four indicators from three categories:

- **Profitability:** Net Profit Margin and Return on Assets.
- **Liquidity:** Current Ratio.
- **Solvency:** Debt-to-Equity.

Each indicator is a calculated relationship between reported values. Trend analysis adds a time dimension by comparing values and indicators across the available periods.

A figure alone does not carry its own meaning. A rise in revenue may be favourable or unfavourable depending on related indicators and context. Analysis therefore has two parts: a numerical part that can be computed exactly, and an interpretive part that must be explained carefully.

1.3 Motivation

The motivation for this project comes from three observations.

1. Statements contain many tables and figures, and locating the relevant values is time-consuming.
2. Computing ratios and comparing periods is repetitive work that follows fixed rules and is suitable for automation.
3. Users without a strong financial background may not know what a ratio or trend implies, even when the calculation is correct.

An automated tool that handles these tasks reduces manual effort, applies calculations consistently, and presents the results in a form non-expert users can understand.

1.4 Problems with Conventional Approaches

In the conventional workflow, a user locates figures in a statement, enters them into another tool, calculates ratios, compares periods, interprets the changes and prepares a report. Each step is performed separately and by hand. This leads to the following problems:

- The workflow is repetitive and time-consuming.
- Manual transcription of figures can introduce errors.
- The calculation method may vary between analysts or between sessions.
- Interpretation depends on the user's financial knowledge.
- Producing a report is a separate effort after the analysis is complete.

These problems are examined in detail in Chapter 3 and Chapter 4.

1.5 Need for the System

The system is needed to bring the above steps into one workflow. The user supplies a statement and receives validated figures, calculated indicators, an explanation of the results, and a dashboard and report. Combining these steps reduces repeated manual entry, keeps the calculation method fixed, and makes the output understandable to users who are not financial specialists.

The system is intended as an educational and analytical tool. It does not replace professional accounting, auditing or financial advice.

1.6 Role of AI and Large Language Models

A Large Language Model (LLM) can generate fluent explanations in simple language, which makes it suitable for communicating the meaning of analytical results. It is less suitable as the primary source of numerical results. It can produce statements that are not supported by its input, and its arithmetic cannot be traced or tested in the way ordinary program code can.

This project therefore assigns the LLM a limited role:

- The application performs extraction, cleaning, validation, ratio calculation and trend calculation.
- The LLM receives only the verified results, together with data-quality notes, and explains them in natural language.
- The LLM is not used to calculate primary values, to supply missing values, or to give guaranteed investment recommendations.

This separation keeps the numbers deterministic and testable, and keeps the LLM within the evidence it is given. The same principle governs every later chapter of this report.

1.7 Project Overview

The proposed system, the *LLM-Assisted Financial Statement Analysis Tool*, is a web application. It accepts a financial statement in PDF or Excel format, extracts the financial data, calculates selected ratios and trends, uses an LLM to explain the verified results, and presents the outcome through a dashboard and a structured report.

The system follows the stages shown in Figure 1.1: Input, Extraction, Analysis, LLM Layer, Insights, Dashboard and Report. Data cleaning, data validation, security and a modular architecture are added to these stages. The application is implemented in Python using the Flask framework, with Pandas, OpenPyXL and PyMuPDF for data and document processing, and HTML, CSS, JavaScript and Chart.js for the user interface. The LLM is accessed through a replaceable LLM/API layer. The technology stack is described in Chapter 7 and the architecture in Chapter 8.

The development plan is incremental. Excel statements, which are well structured, are supported first. Text-based PDF support follows, and more difficult layouts are handled gradually.

1.8 Objectives

The objectives of the project are:

1. To reduce the repetitive manual effort involved in analysing financial statements.
2. To extract financial values from PDF and Excel statements and convert them into structured data.
3. To clean and validate the extracted data, flagging missing or suspicious values instead of inventing them.
4. To calculate Net Profit Margin, Return on Assets, Current Ratio and Debt-to-Equity consistently and deterministically.
5. To compare financial values and indicators across periods.
6. To use an LLM to explain the verified results in simple language, without relying on it for arithmetic.
7. To present the results through a dashboard with summary cards, tables and charts, and to generate a structured report.
8. To protect uploaded documents and credentials through file validation and secure handling of API keys.

1.9 Scope

Within scope

- Upload of financial statements in PDF and Excel formats.
- Extraction from Excel workbooks and from text-based PDF documents.
- Cleaning, unit tracking and validation of the extracted data.
- Calculation of the four indicators named in Section 1.2 and of period changes.
- LLM-generated explanation, insights, a dashboard and a report.

Outside the present scope

- OCR for scanned or image-based PDF statements.
- Document formats other than PDF and Excel.
- Industry-specific indicators and industry benchmark comparison.
- Company-to-company comparison.
- Professional accounting, auditing or investment advice.

The system analyses only the statements supplied to it and does not account for the wider business environment. These and other limitations are discussed further in the later chapters. Section 1.10 gives the organisation of the report.

1.10 Organisation of the Report

Chapter 2 reviews related work and identifies the gap addressed by the project. Chapters 3 to 5 define the problem, describe the existing manual approach and present the proposed system. Chapters 6 and 7 specify hardware and software requirements. Chapter 8 describes the system architecture, Chapter 9 describes the modules, and Chapter 10 presents the UML diagrams.

CHAPTER 2:

LITERATURE SURVEY

2.1 Overview

This chapter reviews work related to the stages of the proposed system. These are financial statement analysis, document processing, PDF and table extraction, OCR and document intelligence, structured financial data, LLM-based document analysis, and hybrid systems that combine deterministic computation with an LLM. Figure 2.1 shows how approaches have progressed. The chapter ends with a comparison of approaches and a statement of the gap addressed by this project.

Only works whose bibliographic details were checked are cited, and they appear in the reference list at the end of the chapter. Topics for which a source is still needed carry an explicit marker.

2.2 Financial Statement Analysis

Ratio analysis and trend comparison are the standard techniques for interpreting financial statements. Profitability, liquidity and solvency indicators are computed from reported values and compared across periods. These techniques are well established and have exact formulas, which makes them suitable for deterministic implementation.

The difficulty lies in the surrounding work: obtaining correct input values, handling different statement layouts and units, and explaining results to non-experts. The remaining sections review how related research addresses these tasks.

2.3 Financial Document Processing

Financial documents present a document-processing problem before they present an analytical one. Values are embedded in tables and text, layouts differ between organisations, and the same quantity may appear under different labels. The need to analyse this volume of material has motivated benchmark work in the financial domain. FinQA, for example, was proposed to support automated analysis of a large corpus of financial reports, and it highlights the complexity of numerical reasoning and heterogeneous data representations in the finance domain [3].

This observation supports a design decision in the present project. Label mapping, unit tracking and validation are treated as explicit processing stages before any ratio is calculated, because incorrect extraction would otherwise propagate into the analysis.

2.4 PDF and Table Extraction

Extracting values from PDF files involves locating tables and recovering their structure. Table extraction is commonly divided into table detection, structure recognition and functional analysis. The PubTables-1M dataset contains nearly one million tables from scientific articles and supports these tasks, and transformer-based detection models trained on it produced strong results for all three [2].

Two observations follow for this project. First, learned table-extraction models require training data and computational resources, whereas the present system uses a library-based approach, PyMuPDF for text-based PDFs. Second, the data in [2] comes from scientific articles, so its results cannot be assumed to transfer to financial statements without evaluation. The limitation that PDF layouts vary and extraction may fail therefore applies to this project, and the document recommends testing extraction independently before any financial calculation.

2.5 OCR and Document Intelligence

Scanned statements contain no machine-readable text, so OCR is needed before extraction. Beyond OCR, document-intelligence models use layout as well as text. LayoutLM jointly models text and layout information across scanned document images for tasks such as information extraction, and reported improved results on form understanding, receipt understanding and document image classification [1].

Models of this kind show that layout carries information that plain text extraction discards. They also need pre-training and fine-tuning, and the evaluation tasks in [1] are forms, receipts and document classification, not financial statement analysis. In the present project, OCR for scanned statements is identified as future scope, and the system is limited to text-based PDFs and Excel files.

2.6 Structured Financial Data and XBRL

XBRL is an XML-based standard for tagging financial reporting data so that values are machine-readable. Where a statement is available in such a form, extraction is largely unnecessary because values and their meanings are explicit. However, many statements that users hold are PDF or spreadsheet documents without such tagging, and the proposed system accepts those formats. XBRL is therefore noted as a complementary route to structured data and as a possible extension. It is not used in this project.

2.7 LLM-Based Document Analysis and Financial LLMs

LLMs have been applied to the financial domain, including domain-specific models. BloombergGPT is a 50-billion-parameter language model trained on a large dataset built from financial data, augmented with general-purpose data [4]. The FinQA study, however, reports that popular large pre-trained models fall far short of expert humans in complex multi-step numerical reasoning over financial data [3].

Two points are relevant here. A financial-domain LLM can improve language understanding of financial text. Numerical reasoning, however, remains a reported weakness of pre-trained models in [3]. This directly motivates keeping calculation outside the LLM in the present system. Also, LLMs are known to produce unintended text that is not supported by the input. Ji et al. survey this problem of hallucination, including measurement and mitigation methods [6]. This project addresses it by constraining the prompt: the LLM receives verified results only, must not invent missing values, and must state uncertainty.

2.8 AI-Assisted Financial Analysis

Systems that use AI for financial analysis range from question answering over reports to explanation of analytical results. The FinQA task expects a model to answer questions over earnings reports by using facts from both text and tables, with annotated reasoning programs provided so that answers are explainable [3]. This shows that explainability of the numerical step is considered important in the literature.

The present system adopts the same concern in a different form. The calculation is performed by deterministic code, and every reported value can be traced to its inputs. The LLM contributes the explanation only.

2.9 Hybrid Deterministic and LLM Systems

Combining an LLM with deterministic computation is an established idea in the LLM literature. PAL (Program-Aided Language models) uses the LLM to read the problem and generate programs as intermediate steps, and delegates the solution step to a runtime such as a Python interpreter, because LLMs often make arithmetic mistakes even when they decompose the problem correctly [5].

In PAL the LLM produces the program and the interpreter computes the answer. In the present project, the computation is fixed in advance: formulas for the four indicators and period change are implemented in application code, and the LLM is given the computed results. The motivation is the same in both cases, which is to keep arithmetic away from the language model. Figure 2.2 contrasts this hybrid pattern with LLM-only and extraction-only patterns.

2.10 Comparative Summary

Ref.	Work	Contribution (as reported)	Relevance to this project	Limitation with respect to this project
[1]	LayoutLM (Xu et al., KDD 2020)	Joint modelling of text and layout for document image understanding	Shows layout information benefits document extraction	Targets scanned forms, receipts and classification; requires model training; not a

				financial analysis system
[2]	PubTables-1M (Smock et al., CVPR 2022)	Large dataset for table detection, structure recognition and functional analysis	Describes table extraction tasks and learned approaches	Built from scientific articles, not financial statements
[3]	FinQA (Chen et al., EMNLP 2021)	Expert-annotated financial question-answering dataset with reasoning programs	Establishes difficulty of numerical reasoning over financial reports	A benchmark, not an end-to-end analysis tool
[4]	BloombergGPT (Wu et al., 2023)	Financial-domain LLM	Shows LLM use in finance	General model for financial NLP tasks, not a statement-analysis pipeline with validation
[5]	PAL (Gao et al., ICML 2023)	LLM generates programs; interpreter computes	Supports separating arithmetic from the LLM	General reasoning tasks, not financial statements
[6]	Hallucination survey (Ji et al., 2023)	Overview of hallucination metrics and mitigation	Justifies grounding and constraining the LLM	Survey of NLG in general, not finance-specific

Table 2.1: Summary of Reviewed Literature

Criterion	LLM-only	Extraction-model-only	Hybrid (proposed)
Extraction from document	By the LLM	By a trained model	By PyMuPDF/Pandas/OpenPyXL, followed by cleaning and validation
Numerical calculation	By the LLM	Not provided	Deterministic application code
Natural-language explanation	Provided	Not provided	Provided by the LLM from verified results

Traceability of numbers	Low	Moderate	High, as values come from tested code
Handling of missing data	Risk of unsupported values	Not applicable	Reported as unavailable, not invented
Dashboard and report output	Not integrated	Not integrated	Integrated

Table 2.2 is a qualitative comparison derived from the design properties of each pattern

2.11 Limitations of Existing Approaches

From the works reviewed, the following limitations are relevant:

1. Learned extraction and document-intelligence models need training data and computational resources, and the datasets reviewed come from other document types [1], [2].
2. Pre-trained LLMs are reported to perform poorly on multi-step numerical reasoning over financial data [3].
3. LLM output can include unsupported statements [6].
4. Domain-specific LLMs and benchmarks address language or reasoning tasks and do not themselves provide an upload-to-report workflow [3], [4].
5. General hybrid methods show the value of delegating computation but are not specified for financial statement analysis [5].

2.12 Research and Engineering Gap

Within the literature reviewed in this chapter, document understanding, numerical reasoning and LLM grounding have been studied largely as separate problems. This project does not claim a new learning algorithm. It addresses an engineering gap: the integration of these concerns into a single, practical, modular workflow for financial statements.

Gap identified	How the proposed system addresses it
Numerical reasoning by LLMs is unreliable [3], [5]	Ratios and trends calculated by deterministic code; LLM does not perform primary arithmetic
LLMs may produce unsupported statements [6]	Prompt restricted to verified results; instructed not to invent missing values and to state uncertainty
Extraction errors propagate into analysis	Separate cleaning and validation stage; missing or suspicious values flagged

Document-understanding research is not packaged as a user workflow [1], [2]	Single pipeline from upload to dashboard and report
Outputs require financial expertise to interpret	Plain-language explanation, strengths and areas to monitor

Table 2.3: Gap Addressed by the Project

2.13 Summary

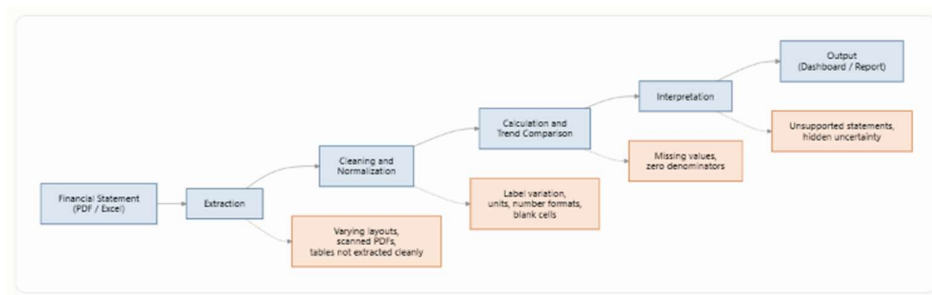
The reviewed work establishes that extracting data from documents is difficult, that LLMs are weak at multi-step numerical reasoning over financial data, and that delegating computation to deterministic code is a recognised way to improve reliability

CHAPTER 3

PROBLEM STATEMENT

3.1 Introduction

A financial statement records an organisation's revenue, expenses, assets, liabilities, equity and profit or loss. It is usually distributed as a PDF or a spreadsheet, and its figures must be extracted, related to one another and interpreted before they inform any decision. This chapter defines the problem addressed by the project. It covers the difficulties of manual analysis, the technical difficulties of automating it, and the risks of introducing a Large Language Model (LLM) into the process. The chapter ends with a precise problem statement and the project objectives.



3.2 Core Problem

To analyse a financial statement, a user must locate the relevant figures, enter them into another tool, calculate ratios, compare periods, interpret the changes and prepare a report. These steps are repetitive. They are also difficult for users who do not have strong financial-analysis knowledge. The same sequence must be repeated for every statement, and the interpretation step depends on the expertise of the person performing it.

The core problem is therefore the absence of a single, consistent workflow that converts a financial statement document into verified numerical results and an understandable explanation.

3.3 Manual and Technical Difficulties

The difficulties can be grouped by the stage at which they occur.

1. **Locating data.** A statement contains many tables and numbers. The required values, such as revenue, net profit, total assets, current assets, current liabilities, total debt and equity, must be identified among them.
2. **Transferring data.** Values are re-entered into another tool before any calculation can be made. This step has to be repeated for each statement and each period.
3. **Calculating indicators.** Ratios such as Net Profit Margin, Return on Assets, Current Ratio and Debt-to-Equity must be computed consistently each time.
4. **Comparing periods.** Values must be compared across available periods to identify direction and significant change.
5. **Interpreting results.** A ratio or a change has to be explained in terms that the intended reader understands.
6. **Reporting.** The results and their interpretation must be assembled into a report.

Each step is manageable in isolation. The difficulty is that the whole sequence is repeated for every statement, and each stage depends on the correctness of the one before it.

3.4 Complexity of Financial Documents

3.4.1 PDF Layout Variation

Financial statements are commonly supplied as PDF files, and PDF layouts vary. A PDF stores the visual position of text on a page and does not necessarily store the logical structure of a statement. Software that reads a PDF must therefore decide which text belongs to which row, column and period. Scanned or image-based PDFs contain no extractable text and require Optical Character Recognition (OCR) before any reading is possible.

3.4.2 Table Extraction Challenges

Statements are organised as tables with line items in rows and periods in columns. Extracting such tables from a PDF may not produce a clean result. Row labels may be separated from their values, columns may be merged or misaligned, and multi-line labels may be split across rows. Excel files are easier to process because their sheets already hold cell-level structure. Even so, a workbook may contain several sheets, and the relevant rows and columns must still be identified.

3.4.3 Terminology Variation

Different statements use different labels for the same item. Different accounting formats also require different mapping rules. A calculation that expects one internal name for a quantity will fail to find it when the source uses another label. Mapping source labels to consistent internal names is therefore required wherever practical.

3.5 Data Normalization Challenges

Extracted values are text, and text is not directly usable in calculation. The following aspects need to be handled before analysis:

- Numeric strings contain commas and currency symbols.
- Values must be converted to numeric types.
- Financial-year labels appear in different forms and must be normalized so that periods can be compared.
- Blank cells and missing values must be handled explicitly.
- Statements may state values in thousands, lakhs, millions or crores. Units must be tracked, because combining values stated in different units gives a wrong ratio or a wrong comparison.

Normalization errors are particularly harmful because they do not stop the computation. The pipeline continues with wrong numbers and still produces results.

3.6 Numerical Calculation Risks

Ratio computation is simple arithmetic, but it is sensitive to the quality of its inputs. A ratio such as the Current Ratio divides one extracted value by another. If the denominator is missing, zero or wrongly extracted, the result is either undefined or incorrect. If a required value is absent, supplying an estimated value gives a number that has no basis in the source document.

There is also an interpretation risk in numerical comparison. An increase in a value is not automatically favourable, and the meaning of a change depends on related indicators and on context. A system that classifies every increase as positive would mislead the reader.

These risks mean that every computed value must be traceable to source values, and that a missing or invalid input must produce an explicit "insufficient data" result and not a fabricated figure.

3.7 Risks of Using an LLM

An LLM can convert numerical results into natural-language explanation, which addresses the difficulty faced by non-expert users. It also introduces risks that must be controlled in the design:

- **Unsupported statements.** An LLM can produce statements that are not supported by the supplied data unless its output is grounded and constrained.
- **Invented values.** When a value is missing, an LLM may supply one and thereby hide an extraction gap.
- **Unreliable arithmetic.** Primary calculation should not depend on an LLM.
- **Hidden uncertainty.** An LLM may present an explanation with unwarranted confidence when the underlying extraction was incomplete or doubtful.
- **Unsuitable advice.** An explanation must not become a guaranteed investment recommendation.

These risks lead to two needs. The first is **deterministic computation**: calculations must be repeatable, independently testable and separate from the LLM. The second is **grounded interpretation**: the LLM must receive only verified results and data-quality notes, and it must be instructed to use only that information, to state uncertainty, and to distinguish calculation from interpretation.

3.8 Propagation of Errors

The difficulties described above do not remain within the stage where they arise. An extraction error enters the cleaning stage, a normalization error enters the calculation, and a wrong calculated value enters the explanation. Incorrect extraction leads to incorrect analysis, and a fluent explanation of a wrong number is more likely to be trusted. Figure 3.1 shows the principal difficulties at each stage of the chain.

Figure 3.1: Propagation of Risks Across the Analysis Chain

The consequence is that the system must validate data before calculation and must report doubtful values and extraction problems to the user. It must not conceal them.

3.9 Formal Problem Formulation

Let S denote an uploaded financial statement file in PDF or Excel format. The required processing can be expressed as a composition of stages:

$$S \rightarrow D \rightarrow (R, T) \rightarrow X \quad (3.1)$$

where D is the cleaned and validated structured data extracted from S , R is the set of calculated financial ratios, T is the set of period-to-period comparisons, and X is the natural-language explanation derived from R , T and the data-quality notes. In this formulation, D , R and T are produced by deterministic processing, and X is produced by the LLM using only the verified content of D , R and T .

3.10 Problem Statement

To design and develop a web-based application that accepts a financial statement in PDF or Excel format and extracts, cleans and validates the financial data from it. The application must calculate selected financial ratios and period-to-period changes through deterministic computation, flagging missing or invalid values instead of inventing them. It must use an LLM only to explain the verified results in simple language, and it must present the explanation, the supporting values and any data-quality warnings through a dashboard and a structured report.

3.11 Project Objectives

Table 3.1 lists the objectives of the project. The project document does not contain a separate list of objectives. These objectives are derived from its statements of purpose and proposed solution.

ID	Objective	Addressed by
O1	Accept financial statements in PDF and Excel formats	File Upload
O2	Extract financial values, labels and tables into structured data	Document Extraction
O3	Normalize and validate the extracted data before analysis	Data Cleaning & Validation
O4	Calculate selected financial ratios deterministically	Financial Analysis
O5	Compare values across available periods	Trend Analysis
O6	Explain verified results in simple language using an LLM	LLM Explanation
O7	Organise observations into strengths and areas to monitor	Insight Generation
O8	Present results through a dashboard and a structured report	Dashboard, Report Generation

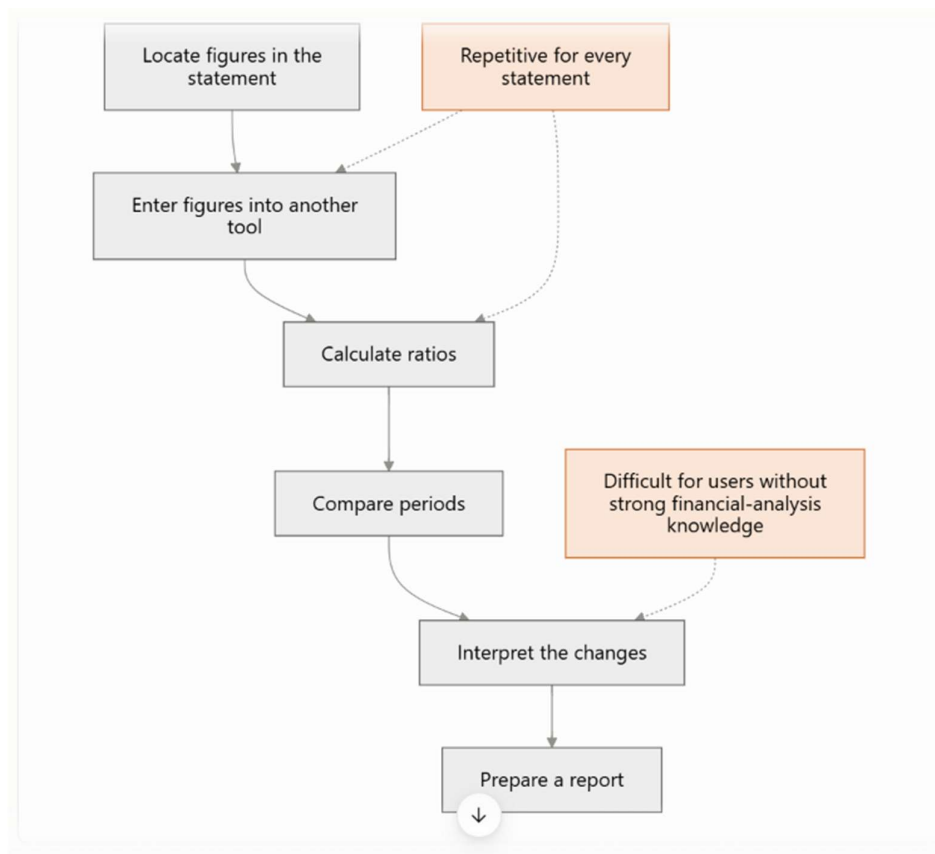
Table 3.1: Project Objectives

CHAPTER 4

EXISTING SYSTEM

4.1 Introduction

The project document does not name specific existing software or research systems. It describes only the manual or fragmented workflow implied by its problem statement. This chapter therefore describes the categories of approach that are relevant to automated financial statement analysis. These are manual analysis, traditional software tools, rule-based systems, structured financial data approaches, PDF and OCR extraction, and direct LLM-based analysis. Each category is described in general terms, together with its advantages and limitations. No individual product, paper or performance result is cited.

**Fig-4.1**

4.2 Manual Analysis

In manual analysis, a person reads the statement, locates the required figures, enters them into another tool, calculates ratios, compares periods, interprets the changes and prepares a report. Figure 4.1 shows this workflow.

Figure 4.1: Manual Financial Statement Analysis Workflow

Advantages. The analyst controls every step and can apply judgement and context. No special software is needed.

Limitations. The process is repetitive and must be performed again for each statement. Values are re-entered by hand, so transcription mistakes are possible. The quality of the interpretation depends entirely on the analyst's expertise, and users without strong financial-analysis knowledge find the process difficult.

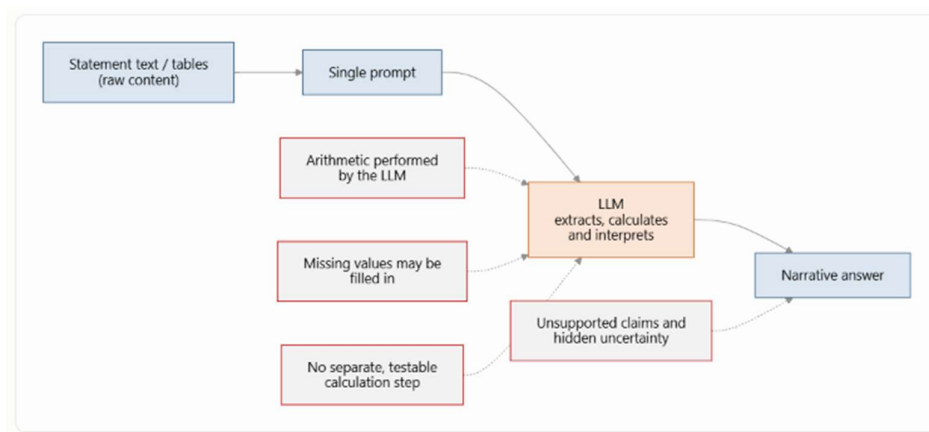


Fig -4.2

4.3 Traditional Software Approaches

General-purpose tools such as spreadsheets and calculators support the calculation stage. The user enters or pastes the values and defines formulas, after which ratios are computed automatically.

Advantages. Calculation is faster and more consistent than hand computation. Formulas can be reused across periods and statements.

Limitations. The user must still locate and enter the source figures. Formulas must be set up and maintained by the user, and the tool provides no explanation of what a result means. The interpretation and report-writing steps remain manual.

4.4 Rule-Based Systems

A rule-based system applies predefined rules to financial values. A rule may be a threshold on a ratio, a pattern for locating a labelled row, or a fixed mapping from source labels to standard names.

Advantages. Behaviour is predictable, repeatable and easy to test. Every output can be traced to a specific rule.

Limitations. Rules cover only the layouts and labels that were anticipated. A statement that uses an unfamiliar label or format needs new rules, so the rule set must grow with each new accounting format. Rules can classify a result, but they do not produce a readable explanation for a non-expert.

4.5 Structured Financial Data Approaches

Where financial data is available in a tagged or tabular machine-readable form, calculation can start directly from named fields without extracting values from a document.

Advantages. No document parsing is required, and field names are consistent. This reduces extraction errors and removes the need for label mapping within that source.

Limitations. The approach depends on the data being supplied in such a form. A statement that arrives only as a PDF or as an ad hoc spreadsheet cannot use it directly. The approach also produces numbers but not a plain-language explanation.

4.6 PDF and OCR Extraction Approaches

Document-processing approaches read text, values and tables from the file. Text-based PDFs can be read directly with a PDF library. Scanned or image-based PDFs require OCR to convert page images into text.

Advantages. Extraction removes manual re-entry and allows statements that exist only as documents to be processed.

Limitations. PDF layouts vary, and tables may not extract cleanly. OCR adds a further source of recognition error on scanned documents. Extraction alone yields text and values, not analysis, and an extraction error that goes unnoticed carries into every later stage (Section 3.8).

4.7 Direct LLM-Based Analysis

In a direct LLM-based approach, the statement text or tables are given to an LLM in a single prompt. The LLM is asked to find the values, compute the ratios and explain the result. Figure 4.2 shows this pipeline and its risk points.

Figure 4.2: Direct LLM-Based Analysis Pipeline and Its Risk Points

Advantages. Explanations are written in natural language and need no formula set-up. One prompt can cover extraction, calculation and explanation, which makes the approach simple to start.

Limitations. Extraction, calculation and interpretation are combined in one step, so the calculation cannot be tested independently. The LLM may perform arithmetic itself, may supply values that are missing, and may make unsupported statements or conceal uncertainty (Section 3.7). If a figure in the explanation is wrong, the user cannot tell whether the cause was extraction, arithmetic or interpretation.

4.8 Summary of Existing Approaches

Existing Approach	Advantages	Limitations
Manual analysis	Full analyst control; applies judgement and context	Repetitive; manual re-entry; depends on analyst expertise; difficult for non-experts
Traditional software (spreadsheets, calculators)	Faster, more consistent calculation; reusable formulas	Source figures still entered by the user; no explanation; interpretation and report remain manual

Rule-based systems	Predictable, testable and traceable	Limited to anticipated layouts and labels; new formats need new rules; no readable explanation
Structured financial data approaches	No document parsing; consistent field names	Requires machine-readable input; no plain-language explanation
PDF/OCR extraction	Removes manual re-entry; handles document-only statements	Layouts vary; tables extract unreliably; OCR adds errors; extraction only, no analysis
Direct LLM-based analysis	Natural-language output; simple to start	Calculation not independently testable; risk of invented values, unsupported claims and hidden uncertainty

Table 4.1: Existing Approach vs Advantages vs Limitations

4.9 Observations

The approaches in Table 4.1 each address part of the problem. Extraction approaches reduce data entry, calculation tools and rule-based systems provide consistency, and LLMs provide readable explanation. No single approach covers the full path from a statement document to a verified and explained result. The approaches that use an LLM do not separate calculation from interpretation. These observations motivate the proposed system described in Chapter 5.

CHAPTER 5

PROPOSED SYSTEM

5.1 Overview

The proposed system is the **LLM-Assisted Financial Statement Analysis Tool**, a web application that accepts a financial statement, extracts and validates its data, calculates selected ratios and period changes, uses an LLM to explain the verified results in simple language, and presents the output through a dashboard and a report.

The system combines document extraction, structured financial analysis, trend identification and LLM-based natural-language explanation in one workflow. Its core design principle is that the application performs the primary numerical calculations and the LLM mainly interprets and explains verified results.

The system is intended for educational and analytical use. It is not a replacement for professional accounting, auditing or financial advice.

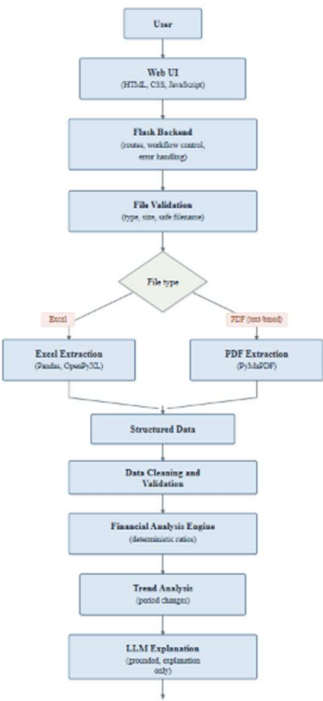


Fig-5.1

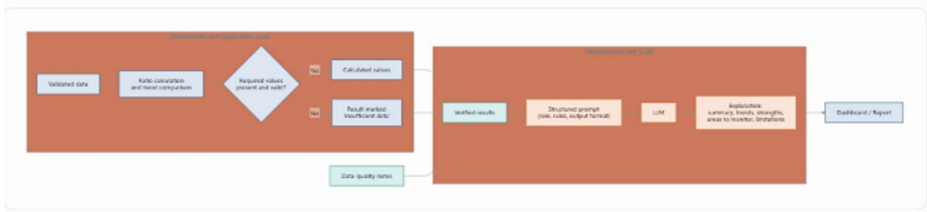
5.2 Scope of the System

Aspect	Description
Input	Financial statement files in PDF or Excel (XLSX) format
PDF support	Text-based PDFs. Scanned PDFs are future scope (OCR)

Indicators	Net Profit Margin, Return on Assets, Current Ratio, Debt-to-Equity, and period change
Output	Dashboard (summary cards, ratio table, charts, explanation, warnings) and a structured report
Out of scope	Professional financial advice, analysis of the wider business environment, and accounting formats that need additional mapping

The system follows a phased build plan. Excel statements are supported first because they are easier to process, and text-based PDF support follows. Difficult layouts are handled gradually.

Figure 5.2



5.3 System Architecture and Workflow

The user interacts with a web interface built with HTML, CSS and JavaScript. A Flask backend receives the request, controls the workflow and handles errors. The uploaded file is first validated for type and size. It is then routed to the Excel extraction path (Pandas and OpenPyXL) or the PDF extraction path (PyMuPDF). Both paths produce structured data, which passes through data cleaning and validation. The validated data is processed by the Financial Analysis Engine and by Trend Analysis. The LLM Explanation layer then explains the verified results. Insight Generation organises the observations, and the Dashboard (Chart.js) presents them. Report Generation produces the final report.

The architecture is organised into eight layers (Presentation, Application, Extraction, Data Preparation, Analysis, AI/LLM, Insights and Output), which are described in detail in Chapter 8. Each module is described in Chapter 9.

5.4 Working Principle

The system operates as an ordered pipeline. Each stage receives the output of the previous stage.

Stage	Description	Technology / Module
1. Upload and File Validation	The user uploads a PDF or Excel file. File type and size are checked, and a safe generated filename is used.	Flask; File Upload

2. Document Processing and Extraction	Excel: sheets, relevant rows and columns are read. PDF: text is extracted, relevant financial sections are identified, and values are parsed. This includes the extraction of tables, labels and values.	Pandas, OpenPyXL; PyMuPDF; Document Extraction
3. Data Normalization	Commas, currency symbols and numeric strings are normalized. Values are converted to numeric types. Financial-year labels are normalized. Blanks are handled. Labels are mapped to consistent internal names where practical. Units (thousands, lakhs, millions, crores) are tracked.	Data Cleaning & Validation
4. Validation	Required values are checked before calculation. Division by zero is prevented. Missing or suspicious values are flagged and not invented. Source information is retained for traceability.	Data Cleaning & Validation
5. Deterministic Financial Computation	Net Profit Margin, Return on Assets, Current Ratio and Debt-to-Equity are calculated by application code. If required values are missing or invalid, an "insufficient data" result is returned.	Financial Analysis Engine
6. Trend Analysis	Values and indicators are compared across available periods. Percentage changes are calculated and direction is classified where appropriate.	Trend Analysis
7. Grounded LLM Interpretation	The verified metrics, ratios, trends and data-quality notes are sent to the LLM in a structured prompt. The LLM explains them in simple language.	LLM Explanation
8. Insight Generation	Observations are organised into strengths, weaknesses, notable changes and areas to monitor.	Insight Generation
9. Dashboard and Report	Summary cards, ratio table, trend charts, explanation, insights and warnings are displayed. The user can generate a structured report.	Dashboard; Report Generation

Table 5.1: Stages of the Proposed Workflow

5.5 Key Design Decisions

Decision	Description
----------	-------------

Separate calculation from interpretation	The Financial Analysis Engine is deterministic and independently testable. The LLM explains verified results and does not compute them.
Structured input to the LLM	The LLM receives verified metrics, ratios, trends and data-quality notes. It does not receive uncontrolled raw document content.
Flag, do not invent	Missing, invalid or suspicious values are flagged. A missing input gives an "insufficient data" result.
Extract Excel first, then text-based PDF	Excel statements are well structured and are implemented first. Text-based PDF support follows, and harder layouts are handled gradually.
Modular architecture	The pipeline is divided into ten modules. Each module is built and tested before the next one.
Replaceable LLM layer	The LLM/API layer is isolated so that the provider can be replaced. The LLM provider is not specified in the project document.
Security by design	File extension and size are validated, uploads are kept in controlled storage out of public paths, and API keys are kept in environment variables. Only necessary information is sent to the external LLM service.
Transparency to the user	Data-quality and extraction warnings are shown on the dashboard.

Table 5.2: Key Design Decisions

5.6 Hallucination Mitigation

Risk	Design measure
Unreliable arithmetic	All primary calculations are performed by the Financial Analysis Engine and Trend Analysis. The LLM does not calculate.
Invented missing values	Missing or invalid inputs produce an "insufficient data" result. The prompt instructs the LLM not to invent missing values.
Unsupported statements	The prompt instructs the LLM to use only the supplied information. Input is limited to verified results.
Hidden uncertainty	Data-quality notes are passed to the LLM, which is instructed to state uncertainty. The output includes a limitations section.
Confusing calculation with opinion	The prompt requires the LLM to distinguish calculation from interpretation.

Misleading reading of changes	Insights consider related indicators and context, because a numerical increase is not automatically positive.
Unsuitable advice	The LLM is not used for guaranteed investment recommendations. The system is described as non-professional financial advice.
Unnoticed extraction errors	Extraction and data-quality warnings are displayed on the dashboard.

Table 5.3: LLM Risks and Corresponding Design Measures

5.7 Existing vs Proposed System

Aspect	Existing approaches (Chapter 4)	Proposed system
Input handling	Figures located and entered manually, or read by a separate extraction step	PDF and Excel files accepted and processed within one workflow
Data preparation	Done by the user or by separate rules	Normalization and validation as part of the pipeline
Calculation	Manual or spreadsheet formulas; in direct LLM use, calculated by the LLM	Deterministic Financial Analysis Engine, independent of the LLM
Missing data	May be ignored, estimated or filled in by an LLM	Flagged; "insufficient data" result is returned
Period comparison	Performed manually	Trend Analysis across available periods
Explanation	Written by the analyst, or generated without grounding	LLM explanation restricted to verified results and data-quality notes
Traceability	Depends on the user	Source information retained; calculation separated from interpretation
Presentation	Separate tools and a manually prepared report	Dashboard with charts and tables, plus a structured report
User expertise required	Strong financial-analysis knowledge for interpretation	Explanations in simple language for non-expert users

Table 5.4: Comparison of Existing Approaches and the Proposed System

5.8 Advantages of the Proposed System

The following advantages follow from the design. They are design-level benefits. The project document reports no measured results.

1. **Reduced repetitive work.** Locating figures, calculating ratios, comparing periods and assembling the report are performed within one workflow.
2. **Consistent calculation.** The same formulas are applied to every statement by application code.
3. **Testability.** The Financial Analysis Engine is deterministic, so it can be tested with known inputs and expected results.
4. **Reduced dependence on the LLM for correctness.** Errors in explanation cannot change the calculated values.
5. **Visibility of data problems.** Extraction and data-quality warnings are shown to the user.
6. **Accessibility for non-experts.** Results are explained in simple language and supported by charts and tables.
7. **Maintainability.** The modular structure allows each module to be built, tested and replaced independently, including the LLM layer.

5.9 Assumptions and Limitations

The proposed system inherits the limitations of its inputs and of its LLM layer:

- PDF layouts vary, so extraction may fail on some documents. Scanned PDFs require OCR, which is not part of the current scope.
- Missing source values limit ratio calculation.
- Incorrect extraction can cause incorrect analysis.
- Different accounting formats require additional mapping rules.
- LLMs can produce unsupported statements unless grounded and constrained. Grounding reduces this risk but does not remove it.
- The system analyses only the supplied statements, not the wider business environment.
- An external LLM API needs internet access and credentials, and may have usage limits.
- The output is not guaranteed professional financial advice.

Authentication, database-backed history, benchmark comparison and other extensions are future scope and are not part of the proposed system described here.

5.10 Summary

The proposed system implements the pipeline Upload → Extraction → Cleaning & Validation → Financial Analysis → Trend Analysis → LLM Explanation → Insights → Dashboard → Report. It addresses the difficulties identified in Chapter 3 by performing numerical work deterministically and by restricting the LLM to explaining verified results. Chapter 6 and Chapter 7 specify the hardware and software requirements, and Chapter 8 describes the architecture in detail.

CHAPTER 6

HARDWARE REQUIREMENTS

6.1 Overview

The LLM-Assisted Financial Statement Analysis Tool is a web application. A user interacts with it through a browser, while a Flask backend performs file handling, document extraction, data cleaning, validation, financial analysis and report generation. The explanation layer is provided by an LLM accessed through an external API.

The project document does not define minimum or recommended hardware specifications, nor the configuration of the machines used for development and testing. This chapter therefore identifies the hardware components relevant to the system's architecture and states their purpose. Numerical specifications are reported as "Not specified in the project document", and no values have been assumed.

6.2 Hardware Requirements

Component	Minimum Requirement	Recommended Requirement	Purpose
Processor (CPU)	Not specified in the project document.	Not specified in the project document.	Runs the Flask backend, PDF/Excel extraction, data cleaning and validation, and the financial and trend calculations.
Memory (RAM)	Not specified in the project document.	Not specified in the project document.	Holds the extracted statement data and tabular structures while an analysis is being processed.
Storage	Not specified in the project document.	Not specified in the project document.	Holds the application code, the controlled uploads/ folder for submitted statements, temporary files, and the SQLite database if history storage is adopted (optional).
Display	Not specified in the project document.	Not specified in the project document.	Renders the upload page, dashboard (summary cards, tables, charts) and report view in a web browser.
Network	Internet connectivity is required when an external LLM API is used. Bandwidth: not specified in the project document.	Not specified in the project document.	Communication with the external LLM service; valid API credentials are also required, and the service may impose usage limits.

GPU	No GPU requirement is stated in the project document.	Not specified in the project document.	Not applicable to the documented design, in which the LLM is accessed through an external API and no local model inference is described.
Client device	A device with a web browser. Hardware specification: not specified in the project document.	Not specified in the project document.	Access to the web interface for uploading statements and viewing the dashboard and report.
Server	Not specified in the project document.	Not specified in the project document.	Hosts the Flask application and the analysis pipeline.

Table 6.1: Hardware requirements of the system

6.3 Component-wise Discussion

Processor and memory. The processing load consists of document extraction, tabular data handling and ratio and trend calculations on the figures of a financial statement. The document does not state processor or memory targets, and no performance measurements are available. Requirements for these components therefore remain unspecified.

Storage. Uploaded statements are placed in controlled storage under generated filenames, are not exposed through public static paths, and temporary files are deleted when no longer needed. Storage must accommodate these files. SQLite is an optional component for analysis history or user data, and the document indicates it is used only if needed.

Display and client. The interface is built with HTML, CSS and JavaScript, and charts are rendered with Chart.js. A browser capable of running these technologies is therefore required on the client side. The specific browser and its version are not specified in the project document.

Network. Network access is needed to reach the external LLM service. The document lists internet access, credentials and usage limits as constraints of external LLM APIs. Sending only the necessary information to the external service is part of the project's privacy approach.

GPU. The document describes no locally hosted model and states no GPU requirement.

6.4 Client–Server and Deployment Considerations

The system follows a client–server arrangement: the browser acts as the client, and the Flask application acts as the server that coordinates the pipeline. Whether the server runs on a local

machine or a cloud platform is not specified in the project document. Cloud deployment is listed as future scope, and no hosting platform or server configuration is documented.

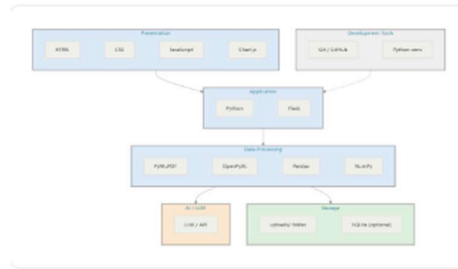
6.5 Hardware Information Not Available in the Project Document

The following items are not available in the project document and are therefore not reported in this chapter:

- processor model, clock speed and core count of the development and test machines;
- installed memory and storage capacity of those machines;
- display resolution;
- network bandwidth;
- any server or cloud instance configuration.

CHAPTER 7

SOFTWARE REQUIREMENTS

Figure 7.1 – Mermaid specification

7.1 Overview

The system is implemented in Python and delivered as a web application. The backend uses Flask, document extraction uses PyMuPDF for PDF files and Pandas/OpenPyXL for Excel files, tabular and numerical processing uses Pandas and NumPy, and the interface is built with HTML, CSS and JavaScript with Chart.js for visualization. The explanation layer is accessed through a replaceable LLM/API component. Versions of the software are not given in the project document, so the Version column of Table 7.1 states this where applicable.

7.2 Software Requirements

Software / Technology	Version	Purpose
Operating system	Not specified in the project document.	Platform on which the application and development tools run.
Python	Not specified in the project document.	Primary programming language for the backend and data processing.
Flask	Not specified in the project document.	Web framework providing routes, workflow control and error handling.
Pandas	Not specified in the project document.	Tabular processing of extracted statement data and Excel sheets.
NumPy	Not specified in the project document.	Numerical operations where required.
OpenPyXL	Not specified in the project document.	Reading and handling of Excel workbooks.
PyMuPDF	Not specified in the project document.	Extraction of text and content from PDF statements.

HTML, CSS, JavaScript	Not applicable (web standards); browser support not specified in the project document.	Browser interface: upload page, dashboard and report view.
Chart.js	Not specified in the project document.	Dashboard charts, such as revenue and profit trends.
LLM / API	Provider, model and version not specified in the project document.	Natural-language explanation and interpretation of verified analytical results.
SQLite (optional)	Not specified in the project document.	Optional storage of history or user data, if needed.
Git / GitHub	Not specified in the project document.	Version control and team collaboration.
Python venv	Distributed with Python; version follows the Python installation, which is not specified in the project document.	Isolation of project dependencies.
Web browser	Not specified in the project document.	Client-side access to the interface.

Table 7.1: Software requirements and technology stack

7.2.1 Technologies Outside the Documented Stack

The following technologies are not part of the stack described in the project document and are therefore not claimed as part of this project:

- **Streamlit:** the user interface is built with Flask together with HTML, CSS and JavaScript.
- **Pydantic:** the document does not name a data-validation library. Validation is described as rules applied by the Data Cleaning and Validation module (file type and size checks, required-value checks, division-by-zero prevention, and flagging of missing or suspicious values).
- **A dedicated table-extraction library:** the document names none. Tables and values are obtained through PyMuPDF (PDF) and Pandas/OpenPyXL (Excel).
- **OCR software:** OCR for scanned statements is future scope.

7.3 Role of the Major Technologies

Python and Flask. Python is the implementation language for the backend, the extraction logic, the analysis engine and the LLM integration. Flask provides the web routes, coordinates the workflow from upload to report, and handles errors. Application settings reside in config.py, and secrets such as API keys are held in environment variables (.env) and are not committed to version control.

Document processing (PyMuPDF, OpenPyXL, Pandas). For Excel statements, the workbook is read through Pandas/OpenPyXL, the relevant sheets, rows and columns are selected, and the values are normalized into structured data. For PDF statements, PyMuPDF extracts the text content, after which the relevant financial sections are identified and the values parsed. Support starts with well-structured Excel files and text-based PDFs. Scanned or image-based PDFs are not covered, since OCR is future scope.

Data processing (Pandas, NumPy). Pandas handles tabular data. NumPy provides numerical operations where required. These support the cleaning, validation, ratio and trend computations, which are kept deterministic and separate from the LLM.

LLM/API. The LLM receives verified metrics, ratios, trends and data-quality notes in a structured prompt and returns an explanation in simple language. It is not used for primary arithmetic or for supplying missing values. The provider and model are not specified in the project document, and the layer is designed to be replaceable.

Frontend and visualization (HTML, CSS, JavaScript, Chart.js). These technologies implement the upload page, dashboard summary cards, ratio table, trend charts and report view.

Storage (uploads folder, SQLite). Uploaded files are kept in controlled storage under generated filenames. SQLite is an optional database for analysis history or user data.

Development tools (Git/GitHub, venv). Git/GitHub support collaboration and versioning among team members, and venv isolates the Python dependencies.

7.4 Software Components and Project Folders

The project follows a modular folder structure in which each component corresponds to a module of the system. Modules are added progressively as the roadmap advances.

Folder / File	Contents	Related Module
app.py	Flask application entry point and routes	User Interface, File Upload, workflow control
config.py	Application configuration	All modules
requirements.txt	Python dependency list	Environment setup
.env	Environment variables, including API credentials	LLM Explanation (secret protection)

extractor/	pdf_extractor.py, excel_extractor.py	Document Extraction
analysis/	ratios.py, trends.py, insights.py	Financial Analysis, Trend Analysis, Insight Generation
llm/	explainer.py	LLM Explanation
reports/	report_generator.py	Report Generation
templates/	index.html, dashboard.html, report.html	User Interface, Dashboard, Report Generation
static/	css/style.css, js/dashboard.js	Dashboard
uploads/	Uploaded statements (controlled storage)	File Upload
tests/	Test cases for each module	Testing

Table 7.2: Software components and corresponding project folders

7.5 Software Version Information

Version numbers of the operating system, Python, Flask, Pandas, NumPy, OpenPyXL, PyMuPDF, Chart.js, SQLite, the browser and the LLM/API are not specified in the project document. They are therefore not stated in this report.

CHAPTER 8

SYSTEM ARCHITECTURE – DESCRIPTION

8.1 System Architecture Overview

The LLM-Assisted Financial Statement Analysis Tool is a web application. A browser-based interface built with HTML, CSS and JavaScript communicates with a Python back end implemented in the Flask framework. The back end is divided into modules, each of which handles one stage of the pipeline defined in the project document: Upload → Extraction → Cleaning & Validation → Financial Analysis → Trend Analysis → LLM Explanation → Insights → Dashboard → Report.

The project document organizes this pipeline into eight layers: Presentation, Application, Extraction, Data Preparation, Analysis, AI/LLM, Insights and Output. Figure 8.1 shows the layers together with the files and technologies that implement them, the storage elements and the one external service. The chapter does not repeat the module-level detail of Chapter 9 or the comparison with existing approaches in Chapter 4.

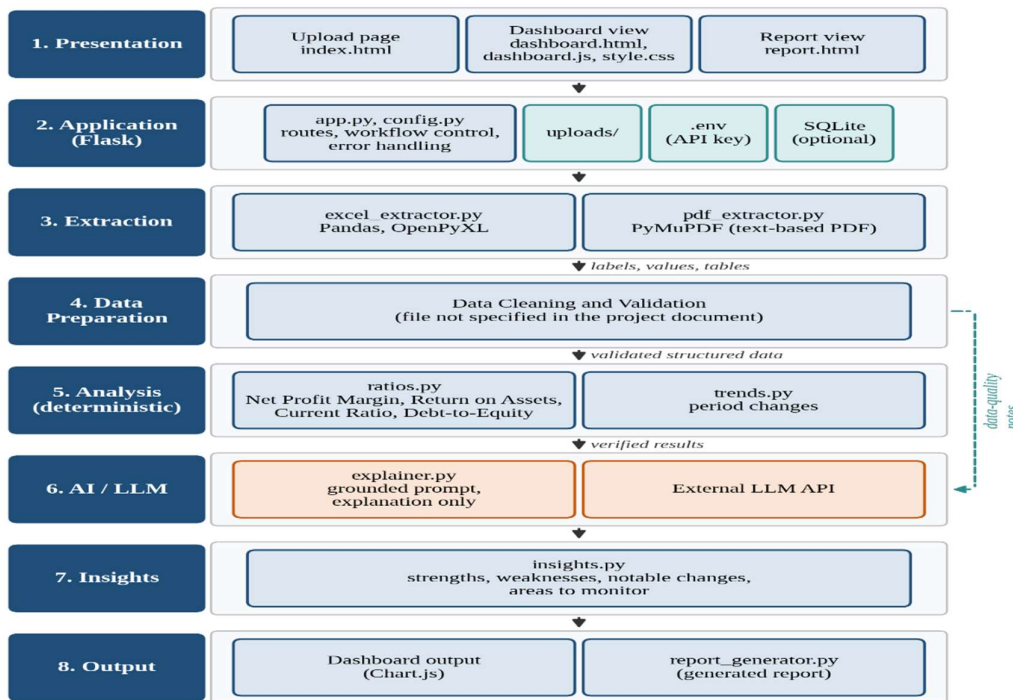


Figure 8.1: Layered Architecture of the System with Implementing Components

The architecture rests on two principles stated in the project document. The first is that the application performs the primary numerical calculations, and the LLM interprets and explains the verified results. Calculation and interpretation are therefore held in separate layers (Analysis and

AI/LLM), and only verified output crosses from the first to the second. The second is that the system does not invent values. Missing or invalid data is flagged and carried forward as "insufficient data", and it is never filled in. Both principles follow from the risks identified in Chapter 3 and are developed in Sections 8.7, 8.8 and 8.10.

The workflow of Chapter 5 maps onto this architecture without change. "Document Processing" and "Table Extraction" are carried out by the Extraction layer. "Data Normalization" and "Validation" are carried out by the Data Preparation layer. "Deterministic Financial Computation" and "Risk/Trend Analysis" are carried out by the Analysis layer, with the risk-oriented output given by the "areas to monitor" of the Insights layer. "Grounded LLM Interpretation" is carried out by the AI/LLM layer. "Dashboard/Output" is produced by the Output layer and displayed by the Presentation layer. The project document accepts Excel files as well as PDF files, so the input stage covers both formats. It also places Insight Generation after the LLM explanation, and the architecture keeps this order.

8.2 Architectural Components

Table 8.1 lists the architectural components with their purpose, input, output and interactions. The file names are those of the recommended project structure in the project document. The document specifies the purpose and flow of each component. It does not define the internal data structures, so inputs and outputs are described by their content and not by their data type.

8.3 Data Flow Through the System

8.3.1 Flow of Data

Figure 8.2 shows how data moves from the uploaded file to the final output. The main flow follows the pipeline order. At each stage, the output of one component is the only input of the next, with three exceptions that run alongside the main chain.

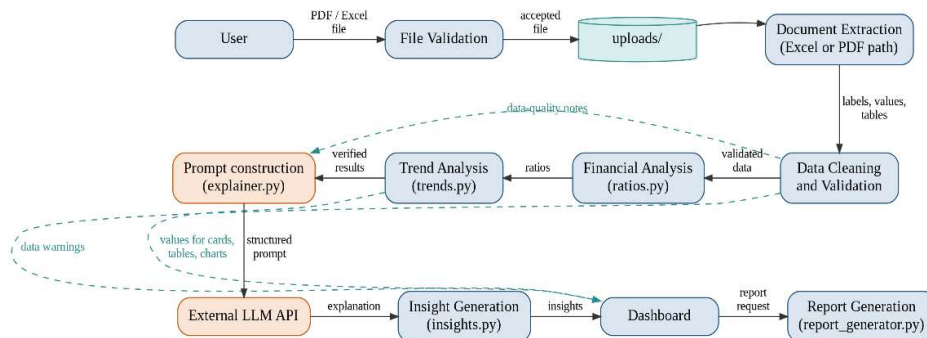


Figure 8.2: End-to-End Data Flow

8.3.2 Dependencies Between Modules

The modules form a dependency chain in which each module depends on the output of its predecessor. Financial Analysis requires validated data and cannot run on raw extraction output. Trend Analysis requires values for more than one period. The LLM Explanation module depends on both the Analysis output and the data-quality notes. Insight Generation depends on the verified results and the explanation. Report Generation depends on the same content as the dashboard.

The Extraction modules are independent of the later modules. For this reason the project document requires extraction to be tested independently, before the financial calculations are built on it, and it fixes a build order from the Flask foundation through to security and final testing. The Presentation layer depends on all layers for its content but contains no financial logic.

The project document does not specify how intermediate results are held between stages, for example in memory, in files or in a database. This is **not specified in the project document**.

8.4 Document Processing Architecture

Document processing begins after a file has passed validation and been stored in uploads/. The Flask application selects the Extraction path from the file type. Figure 8.3 shows both paths.

Figure 8.3: Document Processing Pipeline (Excel and PDF Paths)

The **Excel path** is Excel → Pandas/OpenPyXL → sheets and tables → relevant rows and columns → normalized values → structured data. OpenPyXL handles the workbook, and Pandas handles the tabular processing. NumPy is also listed in the technology stack for numerical operations where required.

The **PDF path** is PDF → PyMuPDF → extracted text → identification of relevant financial sections → parsing of values → structured data. PyMuPDF reads the text content of the PDF. The remaining two steps, identifying the financial sections and parsing the values, are processing steps of the extraction module. The project document does not specify the method used for either step, such as keyword matching, pattern matching or positional parsing. These methods are **not specified in the project document**.

The project document sets out how the two paths are developed. Excel statements are implemented first, because their structure is easier to process. Text-based PDF support follows, and harder layouts are handled gradually. The architecture covers text-based PDFs only. Scanned or image-based PDFs may require OCR, which is listed as future scope, and no OCR component is part of the architecture. Because PDF layouts vary, extraction can fail on some documents. The project document names this as a limitation and requires warnings about extraction problems to be shown on the dashboard.

8.5 Table Extraction Architecture

Table extraction is not a separate module in the project document. It is part of the Document Extraction module, which is defined as reading values, labels and tables from a PDF or Excel file and converting them into structured data.

In the Excel path, tables are read from the workbook's sheets. The relevant rows and columns are then selected from them. A workbook may contain several sheets, and the multi-sheet case is included in the extraction tests of the testing plan. In the PDF path, the project document describes identification of relevant financial sections and parsing of values from the extracted text. It does not describe a dedicated table-detection mechanism. How row labels are associated with period columns in a PDF table is **not specified in the project document**.

The output of this stage is structured data that carries the labels, the values and the associated tables or periods. This output passes to the Data Preparation layer and is not used for calculation directly. The project document warns that tables may not extract cleanly and that incorrect extraction leads to incorrect analysis. The architecture therefore places the cleaning and validation stage between extraction and every calculation, so that incomplete or doubtful table content is flagged and not silently used.

8.6 Data Normalization Architecture

The Data Preparation layer converts extracted content into a consistent representation. The project document specifies six cleaning operations, applied in the order shown in Figure 8.4.

Figure 8.4: Data Normalization and Validation Flow

First, numeric strings are normalized by removing commas and currency symbols. Second, the values are converted to numeric types. Third, financial-year labels are normalized, so that values from different statements or columns refer to periods in a uniform way. Fourth, blank cells and missing values are handled explicitly. Fifth, different source labels are mapped to consistent internal names where practical, so that the Analysis layer can refer to each quantity by a single name. Sixth, the units stated in the source (thousands, lakhs, millions, crores) are tracked, so that values stated in different units are not combined incorrectly.

The result is a structured representation in which each quantity has a consistent internal name, a numeric value, a normalized period label and a recorded unit. This is a description of the required content. The internal field names and the data structure are **not specified in the project document**. The document also does not state whether tracked units are converted to a common unit or recorded for display only.

The project document qualifies label mapping with "where practical" and states that different accounting formats require additional mapping rules. A label that cannot be mapped is treated as missing for the calculations that need it.

8.7 Validation Architecture

Validation takes place at several points in the pipeline. Each point addresses a different risk.

File-level validation. Before extraction, the file type and size are checked, and uploads are stored in controlled storage under a safe, generated filename. The supported types are PDF and Excel. The

size limit is **not specified in the project document**. The testing plan includes an unsupported file and an empty upload as test cases, and these are rejected at this point.

Data-level validation. After cleaning, the Data Preparation layer checks that the values required for each indicator are present before any ratio is calculated. It prevents division by zero and flags missing or suspicious values instead of inventing them. The flow is shown in the lower part of Figure 8.4. A failed check produces two results. The affected indicator is marked "insufficient data", and a data-quality note is recorded. Both results are carried forward, and the pipeline continues for the indicators whose inputs are valid. The project document does not define what makes a value "suspicious". Any numerical rule for this is **not specified in the project document**.

Traceability. The layer retains enough source information for a value to be related back to the document it came from.

Validation of LLM output. The project document tests that the LLM explanation is correct, handles missing data and does not invent values, but it does not specify an automated check of the explanation against the computed values. Such a mechanism is **not specified in the project document**.

Security-related controls. The project document lists the following controls, and no others are described in this chapter. File extensions and size are validated. Safe, generated filenames are used. Uploads are kept in controlled storage and are not exposed through public static paths. API keys are held in environment variables and are never committed to the repository. Temporary files are deleted when no longer needed. Only the information necessary for the explanation is sent to the external LLM service. Sensitive financial content is kept out of logs where practical. The project document does not state at which stage the temporary files are deleted, or what the .env file contains beyond the API key.

8.8 Deterministic Computation Architecture

The Analysis layer performs all numerical computation in application code. It is deterministic, which means that the same validated inputs always give the same results, and it is independent of the LLM. This makes it testable with known inputs and expected outputs, as the testing plan requires.

ratios.py calculates the four indicators in Table 8.3. The formulas are those of Chapter 9 (Equations 9.1 to 9.4).

If a required input is missing or invalid, or a divisor is zero, the module returns an unavailable or "insufficient data" result. It does not estimate the value. This rule is the point at which the architecture prevents invented numbers from entering the later stages.

The boundary between deterministic computation and LLM interpretation is the following. The Analysis layer produces numbers: ratios, period changes and their direction. The LLM layer produces text about those numbers. The LLM does not calculate, and nothing it returns is used to

alter a calculated value. The values shown on the dashboard come from the Analysis layer. Chapter 5 gives the reasoning for this separation, and Figure 5.2 illustrates it.

8.9 Risk and Trend Analysis Architecture

Trend Analysis is implemented in `trends.py`. It compares financial values and indicators across the available periods. The project document defines the indicator as Period Change, which compares a value between available periods. It states that the system can calculate percentage changes, classify the direction where appropriate and display charts. The project document refers to percentage changes but states no formula for them. The formula used is defined in Chapter 9.

Trend Analysis requires values for at least two periods. The chart output for revenue and profit trends is drawn by `Chart.js` on the dashboard, from values supplied by this module. The LLM is then given the verified changes and explains the observed trend.

The classification of direction is subject to a caution in the project document: a numerical increase is not automatically positive, and interpretation must consider related indicators and available context. The module therefore reports changes and does not treat every increase as favourable.

The project document does not define a separate risk-analysis module, risk score or risk metric, and none is introduced here. The risk-oriented output of the system is the "areas to monitor" produced by Insight Generation, together with the data-quality warnings. The criteria by which an item is placed under "areas to monitor", such as thresholds, are **not specified in the project document**. Likewise, the document does not state whether the insight rules, the LLM, or both produce that classification.

8.10 LLM Integration Architecture

Position in the pipeline. The LLM Explanation module (`llm/explainer.py`) follows Trend Analysis and precedes Insight Generation. It is the only module that communicates with an external service. The project document describes the LLM/API layer as replaceable, which isolates the dependency on any provider.

Input and context. The LLM receives structured information and not raw document content. According to the project document, the structured prompt contains the following: the role (a financial-analysis explanation assistant); the input (verified metrics, ratios, trends and data-quality notes); the task (to explain the results in simple language); the rules; and the output format. This content is the grounding context. The LLM is given nothing else about the statement. The same principle governs what leaves the local system, since only the necessary information is sent.

Expected output. The LLM returns a natural-language explanation with an executive summary, key trends, strengths, areas to monitor and limitations. The explanation is passed to Insight Generation, the dashboard and the report. Table 8.4 summarizes the interface.

Control of hallucination. Grounding is achieved by four mechanisms in the architecture, which Table 5.3 presents as risk and measure pairs. The first is that the LLM is placed after the

deterministic calculations and receives only their verified output. The second is that missing inputs reach the LLM as "insufficient data" together with data-quality notes, so there is nothing for it to supply. The third is that the prompt rules require use of supplied information only and a statement of uncertainty. The fourth is that the explanation is shown beside the calculated values and the extraction warnings on the dashboard, so that the user can compare them. The project document accepts that LLMs can still produce unsupported statements, so these measures reduce the risk and do not remove it.

Calculations kept outside the LLM. All four ratios, all period changes and all percentage changes are computed in `ratios.py` and `trends.py`. The values in the summary cards, the ratio table and the charts come from the same modules. The LLM calculates nothing.

8.11 Output and Dashboard Architecture

The Output layer produces the dashboard and the generated report. The Presentation layer displays them, using `dashboard.html` with `dashboard.js` and `Chart.js`, and `report.html`.

The dashboard contains the following sections, as specified in the project document: the analysis and upload status; summary cards for revenue, profit, assets, liabilities and selected ratios; a financial ratio table; revenue and profit trend charts; an assets-versus-liabilities chart where appropriate; the AI explanation; strengths and positive observations; areas to monitor; data-quality and extraction warnings; and a report generation action. `Chart.js` is the only charting technology specified. Tables and summary cards come from the same verified values.

The report is generated by `reports/report_generator.py` when the user triggers the report generation action. It contains twelve sections: Executive Summary, Financial Overview, Extracted Financial Data, Profitability Analysis, Liquidity Analysis, Solvency Analysis, Trend Analysis, LLM Explanation, Key Insights, Areas to Monitor, Data/Extraction Limitations and Conclusion. The testing plan requires the report to show correct values and a clear AI section. The file format in which the report is exported, such as PDF, HTML or Word, is **not specified in the project document**. The project document names only the `report.html` template, and it lists "advanced report formats" as future scope.

No screenshots of the implemented interface are included in this chapter, because none are available in the project document.

8.12 End-to-End Processing Sequence

1. The user uploads a PDF or Excel statement through the upload page.
2. The Flask application receives the file and invokes file validation. The file type and size are checked, and a safe, generated filename is assigned. An unsupported or empty upload is rejected.
3. The accepted file is stored in `uploads/`.

4. The application selects the Excel path (Pandas, OpenPyXL) or the PDF path (PyMuPDF) from the file type, and the extractor returns structured data.
5. The Data Preparation layer normalizes values, year labels, labels and units and checks the required values. Missing or suspicious values are flagged and recorded as data-quality notes.
6. ratios.py calculates Net Profit Margin, Return on Assets, Current Ratio and Debt-to-Equity. An indicator whose inputs are missing, invalid or zero-divisor is marked "insufficient data".
7. trends.py compares values and indicators across the available periods.
8. explainer.py builds a structured prompt from the verified results and the data-quality notes and sends it to the external LLM API. The explanation is returned.
9. insights.py organizes strengths, weaknesses, notable changes and areas to monitor.
10. The dashboard presents the summary cards, ratio table, Chart.js charts, explanation, insights and warnings.

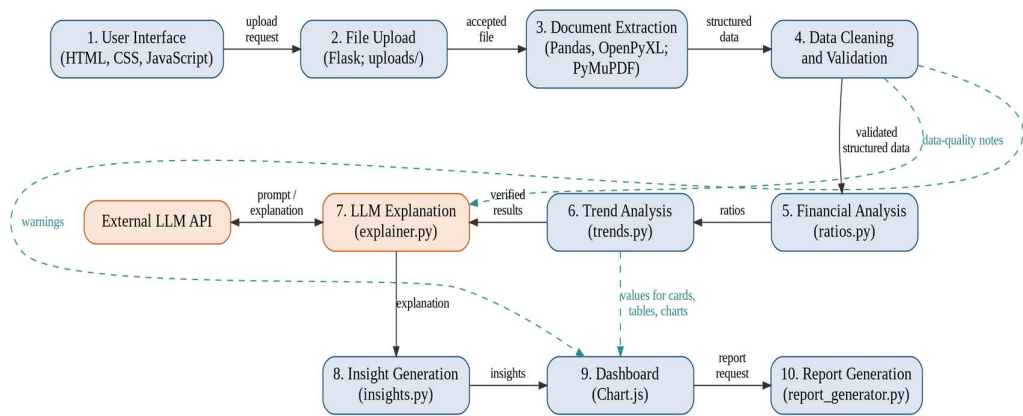
CHAPTER 9

MODULES – DESCRIPTION

9.1 Module Overview

The project document divides the LLM-Assisted Financial Statement Analysis Tool into ten modules: User Interface, File Upload, Document Extraction, Data Cleaning & Validation, Financial Analysis, Trend Analysis, LLM Explanation, Insight Generation, Dashboard and Report Generation. Each module corresponds to one stage of the pipeline Upload → Extract → Clean → Validate → Calculate → Compare → Explain → Visualize → Report. The modules are built and tested one at a time, in the order of the project roadmap, so that each module has a verified input before the next one is added. The modules work as a chain. The User Interface and File Upload modules receive the statement. Document Extraction converts it into structured data, and Data Cleaning & Validation prepares that data for calculation. Financial Analysis and Trend Analysis perform all numerical work. LLM Explanation then explains the verified results. Insight Generation organizes the

observations, and the Dashboard and Report Generation modules present them to the user. Three flows run alongside the main chain: the data-quality notes reach the LLM Explanation module, and the warnings and the numerical values reach the Dashboard directly. Figure 9.1 shows the modules and the direction of data flow between them. The layer-level view of the same structure is given in Chapter 8.



9.2 Module Identification

Table 9.1 lists the modules exactly as named in the project document. The document does not define a separate module for table extraction or for risk analysis. Table extraction is part of the Document Extraction module. The trend-oriented part of "Risk/Trend Analysis" is the Trend Analysis module, and its risk-oriented part is the "areas to monitor" output of the Insight Generation module. The stages "Document Processing", "Data Normalization" and "Validation" of the proposed workflow (Chapter 5) correspond to Document Extraction and to Data Cleaning & Validation. The stage "Grounded LLM Interpretation" corresponds to the LLM Explanation module. File validation is performed within the File Upload workflow, as described in Section 9.3.2.

9.3 Detailed Module Descriptions

Each module is described under the same headings: purpose, input, processing, output, internal operations, interaction with other modules, validation and exception handling, and technical dependencies. Where the project document gives no detail, the text says so. The file names are those of the recommended project structure in the project document.

9.3.1 User Interface

Purpose. The module provides the browser interface through which the user uploads a statement and views the results.

Input. The file selected by the user and the user's requests, such as an upload or a report request. The module also receives the results returned by the application.

Processing. The module presents the upload page, the dashboard and the report view. It contains no financial logic. It displays values that the Analysis modules have computed and does not compute or change them.

Output. An upload request to the Flask application, and the rendered upload page, dashboard and report view.

Internal operations. The module is implemented by the templates `index.html`, `dashboard.html` and `report.html`, the stylesheet `static/css/style.css` and the script `static/js/dashboard.js`.

Module interaction. The module submits the uploaded file to File Upload. It displays the output of the Dashboard and Report Generation modules.

Validation and exception handling. The project document does not state whether the interface performs checks of its own. The documented file checks are made during upload (Section 9.3.2).

Technical dependencies. HTML, CSS and JavaScript. Chart.js is used for the charts shown on the dashboard.

9.3.2 File Upload

Purpose. The module receives the uploaded file and stores it safely. It is the entry point of the pipeline and ensures that only supported files reach extraction.

Input. A financial statement file in PDF or Excel (XLSX) format.

Processing. The module checks the file type and size, assigns a safe, generated filename and stores the accepted file in the `uploads/` directory. The Flask application then selects the Excel or the PDF extraction path according to the file type. The master architecture of the project document lists this check as a separate "File Validation" stage, and the validation rules list it among the checks performed before analysis. This chapter describes it within File Upload, because the project document's list of modules has no separate module for it.

Output. An accepted file stored in controlled storage, or a rejection message for an unsupported or empty file.

Internal operations. Upload handling is implemented in app.py, with configuration in config.py. Uploaded files are kept in controlled storage and are not exposed through public static paths. Temporary files are deleted when they are no longer needed.

Module interaction. The module receives the file from the User Interface and passes the accepted file to Document Extraction.

Validation and exception handling. The testing plan includes a valid PDF or XLSX file, an unsupported file and an empty upload. The Flask application handles errors and reports them to the user. The maximum file size and the point at which temporary files are deleted are **not specified in the project document**.

Technical dependencies. Python and Flask.

9.3.3 Document Extraction

Purpose. The module reads the financial values, labels and tables from the uploaded statement and converts them into structured data that software can process. Table extraction is part of this module and not a separate one.

Input. The stored PDF or Excel file.

Processing. The module has two paths. For an Excel file, it reads the workbook with Pandas and OpenPyXL, locates the sheets and tables, and selects the relevant rows and columns. For a text-based PDF, it extracts the text with PyMuPDF, identifies the relevant financial sections and parses the values. The two paths are shown in Figure 8.3. Excel statements are supported first because their structure is easier to process, and text-based PDF support follows. More difficult layouts are handled gradually.

Output. Structured data consisting of the labels, values and tables found in the statement.

Internal operations. The Excel path is implemented in extractor/excel_extractor.py and the PDF path in extractor/pdf_extractor.py. The methods used to identify financial sections, to parse values and to associate row labels with period columns in a PDF table are **not specified in the project document**.

Module interaction. The module receives the accepted file from File Upload and passes its output to Data Cleaning & Validation. Its output is never used for calculation directly.

Validation and exception handling. PDF layouts vary and tables may not extract cleanly, so extraction can fail. The project document therefore requires the module to be tested independently, with a simple Excel file, a multi-sheet Excel file and a text PDF, before the financial calculations are built on it. Scanned or image-based PDFs may require OCR, which is future scope. Extraction problems are reported as warnings on the dashboard.

Technical dependencies. Pandas, OpenPyXL and PyMuPDF. NumPy is listed in the technology stack for numerical operations where required.

9.3.4 Data Cleaning & Validation

Purpose. The module makes the extracted data consistent and checks it before any calculation. It ensures that missing or doubtful values are flagged and never invented.

Input. The structured data produced by Document Extraction.

Processing. The module applies the cleaning operations in the order shown in Figure 8.4. It normalizes commas, currency symbols and numeric strings, converts values to numeric types, normalizes financial-year labels, handles blank cells and missing values, maps different labels to consistent internal names where practical, and tracks the units stated in the source (thousands, lakhs, millions, crores). It then validates the data. It checks that the required values are present before any ratio is calculated, prevents division by zero, and flags missing or suspicious values. It retains enough source information for traceability.

Output. Validated structured data and a set of data-quality notes that describe any missing, invalid or suspicious values.

Internal operations. The project document assigns no source file to this module, and its internal field names and data structures are **not specified in the project document**. The document also does not state whether units are converted to a common unit, nor what makes a value "suspicious".

Module interaction. The module receives structured data from Document Extraction. It passes the validated data to Financial Analysis, the data-quality notes to LLM Explanation, and the warnings to the Dashboard.

Validation and exception handling. The tests for this module cover comma-formatted values, currency symbols, blanks, a missing denominator, an invalid number and a missing period. A failed check does not stop the pipeline. The affected indicator is later reported as "insufficient data", and a data-quality note is recorded.

Technical dependencies. The project document names no library specific to this module. Pandas and NumPy are listed for tabular and numerical processing.

9.3.5 Financial Analysis

Purpose. The module is the numerical core of the system. It calculates the selected financial ratios deterministically and independently of the LLM, so that every calculated value can be tested and traced to its source values.

Input. The validated structured data: Net Profit, Revenue, Total Assets, Current Assets, Current Liabilities, Total Debt and Equity.

Processing. The module calculates four indicators, which belong to three categories. Net Profit Margin and Return on Assets are profitability indicators:

$$\text{Net Profit Margin (\%)} = (\text{Net Profit} \div \text{Revenue}) \times 100$$

$$\text{Return on Assets (\%)} = (\text{Net Profit} \div \text{Total Assets}) \times 100$$

Current Ratio is a liquidity indicator, and Debt-to-Equity is a solvency indicator:

$$\text{Current Ratio} = \text{Current Assets} \div \text{Current Liabilities}$$

$$\text{Debt-to-Equity} = \text{Total Debt} \div \text{Equity}$$

If a required source value is missing or invalid, or a divisor is zero, the module returns an unavailable or "insufficient data" result. It does not estimate a value.

Figure 9.2 shows this logic together with the trend calculation of Section 9.3.6.

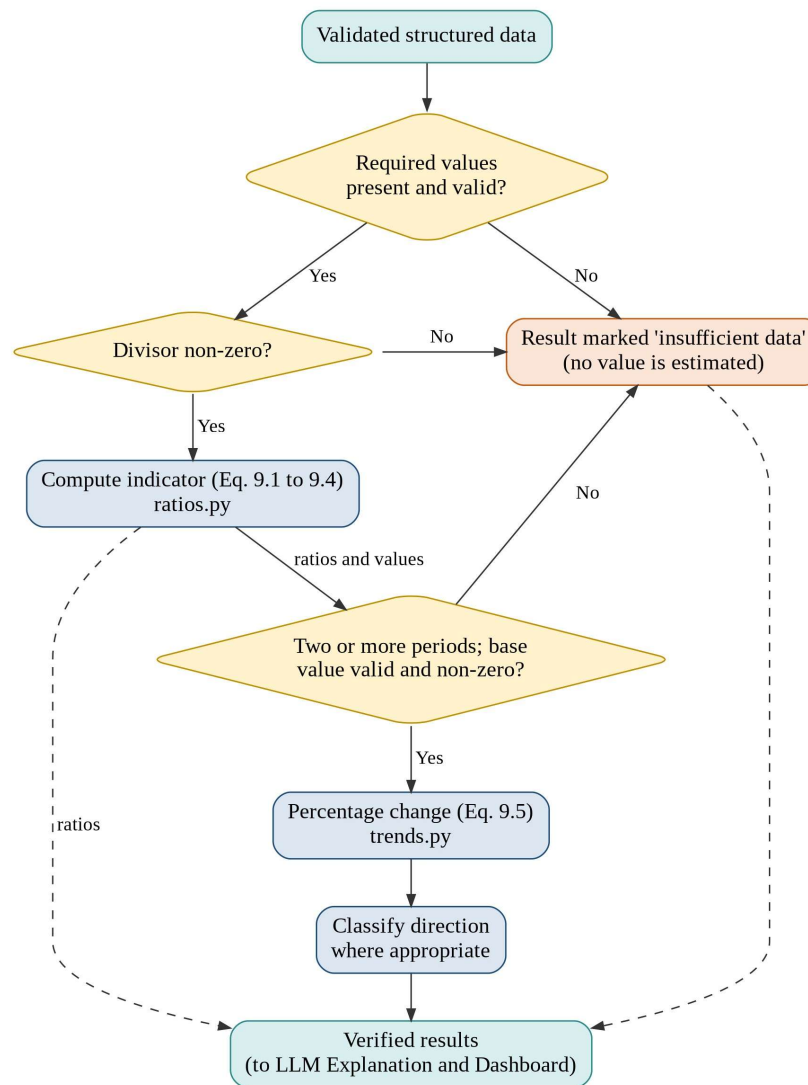


Figure 9.2: Financial Computation Workflow (Financial Analysis and Trend Analysis)

Output. The values of the four ratios for each period for which the inputs are valid, and an "insufficient data" marker for every other indicator.

Internal operations. The module is implemented in `analysis/ratios.py`. The calculations are plain arithmetic on validated values. No statistical or machine-learning method is specified in the project document.

Module interaction. The module receives validated data from Data Cleaning & Validation. Its results are used by Trend Analysis, LLM Explanation and the Dashboard.

Validation and exception handling. The testing plan checks the ratios with known inputs and expected results. The checks of required values and of zero divisors are those described in Section 9.3.4.

Technical dependencies. Python. NumPy is listed for numerical operations where required.

9.3.6 Trend Analysis

Purpose. The module compares financial values and indicators across the available periods, so that the direction and size of change can be shown and explained.

Input. The validated values and calculated ratios for the available periods.

Processing. The project document defines the indicator Period Change as a comparison of a value between available periods. It states that the system can calculate percentage changes, classify the direction where appropriate and display charts. The document gives no formula for the percentage change. This report uses the conventional definition, where V is the value in a period and V_{t-1} is the value $t-1$ in the preceding period:

$$\text{Percentage Change (\%)} = ((V_t - V_{t-1}) \div V_{t-1}) \times 100 \quad (9.5)$$

A change can only be calculated when values for at least two periods are available. Applying the documented rule against division by zero, a change whose base value is zero or missing is reported as "insufficient data". The project document cautions that a numerical increase is not automatically positive, so the module reports the size and direction of a change and leaves its significance to the interpretation stages, which consider related indicators and context.

The project document illustrates the module with an example in which revenue is 8, 9 and 11 Cr and profit is 1.0, 1.3 and 1.6 Cr in 2023, 2024 and 2025. The currency symbol is not legible in the project document, so only the unit "Cr" is used here. Table 9.2 applies Equations 9.1 and 9.5 to these values. The project document gives the values only. The percentages in Table 9.2 and Figure 9.3 are calculated for this report, and they are not outputs of the implemented system.

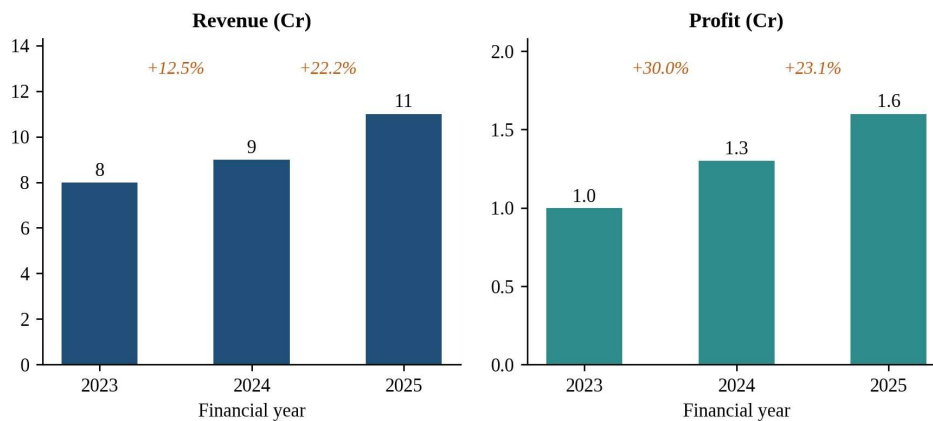


Figure 9.3: Illustrative Revenue and Profit Trends (Example Values from the Project Document)

Output. Period changes and their direction for the values and indicators, and the data from which the revenue and profit trend charts of the dashboard are drawn.

Internal operations. The module is implemented in `analysis/trends.py`. The rule for classifying direction, and any threshold for a "significant" change, are **not specified in the project document**.

Module interaction. The module receives validated data and ratios from Financial Analysis and Data Cleaning & Validation. It supplies verified results to LLM Explanation and numerical values to the Dashboard.

Validation and exception handling. The testing plan covers two-period and three-period comparisons. A missing period is a validation case of the preceding module.

Technical dependencies. Python. Chart.js draws the charts on the dashboard, from the values supplied by this module.

9.3.7 LLM Explanation

Purpose. The module converts the verified analytical results into a natural-language explanation that a user without strong financial-analysis knowledge can follow. It interprets the results and does not calculate them.

Input and grounding information. The module receives verified metrics, ratios, trends and data-quality notes. It does not receive uncontrolled raw document content. This content is the only grounding information that the LLM is given.

Processing. The module builds a structured prompt with five parts, as defined in the project document. The role is a financial-analysis explanation assistant. The input is the verified metrics, ratios, trends and data-quality notes. The task is to explain the results in simple language. The rules are to use only the supplied information, not to invent missing values, to state uncertainty and to distinguish calculation from interpretation. The output format is an executive summary, key trends, strengths, areas to monitor and limitations. The module sends the prompt to the external LLM API and receives the explanation. The tasks suited to the LLM are to explain the meaning of a ratio, to summarize trends, to explain why a change may deserve attention using the supplied evidence, to combine several verified findings into an executive summary and to organize observations into strengths and areas to monitor.

Output. A natural-language explanation with the sections listed above.

Internal operations. The module is implemented in `llm/explainer.py`. The project document describes the LLM/API layer as replaceable, so the provider can be changed without affecting the calculations. The LLM provider, the model, the format of the exchange with the API, the final prompt text and any generation parameters, such as temperature or token limits, are **not specified in the project document**. The document describes no fine-tuning, retrieval-augmented generation or vector database, and none is part of the module.

Position in the pipeline. The module follows Trend Analysis and precedes Insight Generation. Figure 5.2 shows the grounded flow of verified results into the LLM, and Table 8.4 summarizes its interface.

Hallucination mitigation. The project document specifies the following measures. The LLM does not perform the primary arithmetic, because all calculations are made by Financial Analysis and Trend Analysis. Missing values are passed on as "insufficient data" with data-quality notes, and the prompt forbids inventing them. The prompt restricts the LLM to the supplied information and requires it to state uncertainty. The LLM must not hide uncertainty or extraction errors and must not give guaranteed investment recommendations. The explanation appears on the dashboard beside the calculated values and the extraction warnings. Table 5.3 lists these measures as pairs of risk and design measure. The project document accepts that grounding reduces the risk of unsupported statements without removing it.

Module interaction. The module receives verified results from Trend Analysis and Financial Analysis, and data-quality notes from Data Cleaning & Validation. It passes the explanation to Insight Generation, the Dashboard and Report Generation.

Validation and exception handling. The testing plan checks that the explanation is correct, that missing data is handled and that no values are invented. The external API needs internet access and credentials and may have usage limits. The behaviour of the module when the service is unavailable is **not specified in the project document**. An automated comparison of the explanation with the computed values is also not specified.

Technical dependencies. Python and an external LLM API.

9.3.8 Insight Generation

Purpose. The module organizes the findings into a form that the user can read at once: strengths, weaknesses, notable changes and areas to monitor.

Input. The verified results and the explanation.

Processing. The module groups the observations under the headings above. The "areas to monitor" output is the risk-oriented output of the system, because the project document defines no separate risk-analysis module, risk score or risk metric.

Output. The insights shown on the dashboard and in the report.

Internal operations. The module is implemented in `analysis/insights.py`. The project document mentions "insight rules" among the responsibilities of the analysis team member. It does not state whether the insights are produced by rules, by the LLM or by both, and it does not give the criteria for placing an item under "areas to monitor". Both points are **not specified in the project document**.

Module interaction. The module receives its input from LLM Explanation and supplies the Dashboard and Report Generation.

Validation and exception handling. The testing plan has no separate test area for this module. The project document cautions that interpretation of a numerical change should consider related indicators and context.

Technical dependencies. Python.

9.3.9 Dashboard

Purpose. The module presents the results to the user as summary values, tables, charts and explanations.

Input. The verified results, the LLM explanation, the insights and the data-quality and extraction warnings.

Processing. The module arranges the results into the sections listed in the project document: the analysis and upload status; summary cards for revenue, profit, assets, liabilities and selected ratios; a financial ratio table; revenue and profit trend charts; an assets-versus-liabilities chart where appropriate; the AI explanation; strengths and positive observations; areas to monitor; data-quality and extraction warnings; and a report generation action. All numerical content comes from the Analysis modules and not from the LLM. Table 9.3 shows the source module of each section, and Figure 9.4 shows the sections schematically.

Table 9.3: Dashboard Sections and Their Source Modules

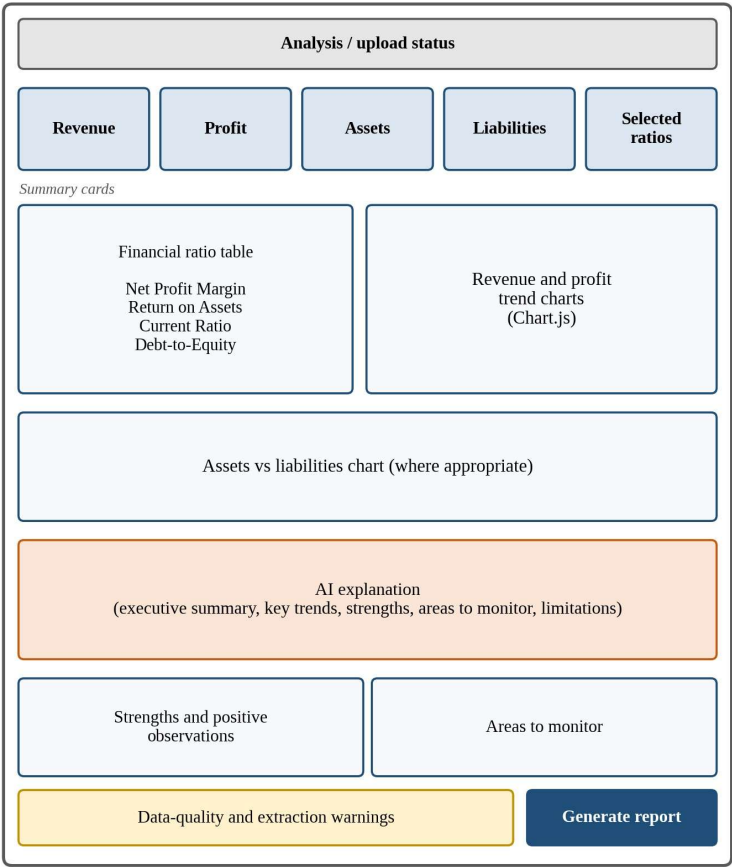


Figure 9.4: Schematic of the Dashboard Sections (Arrangement Illustrative; Not a Screenshot)

Output. The dashboard view, and the report request issued when the user selects the report generation action.

Internal operations. The module is implemented in dashboard.html and static/js/dashboard.js, with Chart.js for charts. The project document lists the sections but does not specify their arrangement, so Figure 9.4 is a schematic and not a representation of the implemented layout. No screenshots are available in the project document.

Module interaction. The module receives content from Financial Analysis, Trend Analysis, LLM Explanation, Insight Generation and Data Cleaning & Validation. It is displayed through the User Interface and triggers Report Generation.

Validation and exception handling. The testing plan checks that the dashboard shows correct numbers, labels and charts. Warnings about extraction and data quality are displayed so that doubtful values are visible to the user.

Technical dependencies. HTML, CSS, JavaScript and Chart.js.

9.3.10 Report Generation

Purpose. The module creates the structured final report from the verified results of the analysis.

Input. The same verified content that the dashboard presents.

Processing. When the user selects the report generation action, the module compiles a report of twelve sections: Executive Summary, Financial Overview, Extracted Financial Data, Profitability Analysis, Liquidity Analysis, Solvency Analysis, Trend Analysis, LLM Explanation, Key Insights, Areas to Monitor, Data/Extraction Limitations and Conclusion. This report is the output of the software and is distinct from the chapters of this project report.

Output. The generated report.

Internal operations. The module is implemented in `reports/report_generator.py`, with the template `report.html`. The format in which the report is exported is **not specified in the project document**, and "advanced report formats" are listed as future scope.

Module interaction. The module is triggered from the Dashboard and uses the same content. The result is displayed through the User Interface.

Validation and exception handling. The testing plan checks that the report contains correct values and a clear AI section, so that the LLM-generated text can be distinguished from the calculated results.

Technical dependencies. Python and HTML (`report.html`).

9.4 Classification of Data and Results in the Financial Processing Modules

The financial processing modules handle values in several states. The state of a value determines how far it can be trusted and which module may use it. Table 9.4 separates these states and identifies the module that produces each. The separation matters because the LLM operates on the last states only and never produces a calculated value.

9.5 Module Interaction and Dependencies

The modules depend on each other in the order of Figure 9.1. Financial Analysis cannot run on data that has not passed Data Cleaning & Validation. Trend Analysis needs the values for several periods. LLM Explanation needs both the verified results and the data-quality notes. Insight Generation needs the verified results and the explanation. The Dashboard and Report Generation need the output of all preceding modules. The Document Extraction module is independent of the modules that follow it, which allows it to be tested alone. The User Interface depends on all modules for its content but contains no financial logic. Table 9.5 lists the files, the specified technologies and the modules from which each module receives its input.

Two storage elements support the modules. The uploads/ directory holds uploaded files for File Upload and Document Extraction. SQLite is named in the project document as an optional database for history or user data, and it is not part of any module described in this chapter. The only external service is the LLM API used by the LLM Explanation module.

9.6 Summary

The project consists of ten modules that follow the pipeline of the project document. The Extraction and Data Preparation modules turn the uploaded statement into validated structured data. The Financial Analysis and Trend Analysis modules compute every number deterministically. The LLM Explanation module explains only verified results, and the Insight Generation, Dashboard and Report Generation modules present them. This division keeps calculation and interpretation separate in every module. Chapter 10 presents the UML diagrams of the same structure.

UML FIGURE PLAN AND VALIDATION NOTES

Author reference only. This part is not report text; remove it before inserting Chapter 10 into the report.

Figure No.	UML	Status	Purpose	Actors /	Placement
------------	-----	--------	---------	----------	-----------

	Diagram			Components	
Figure 10.1	Use Case	Required	Documented user interactions with the system	User; LLM Service (external)	10.2
Figure 10.2	Activity	Required	End-to-end workflow from upload to dashboard and report	Not actor-specific	10.3
Figure 10.3	Sequence	Required	Time-ordered hand-offs between modules for one analysis run	User, pipeline modules, LLM Service	10.4
None	Class	Not applicable	Classes, attributes, operations and inheritance are not specified	None	10.5 (explanation only)
Figure 10.4	Component	Required	Components, dependencies and documented artifacts	Modules, Flask Backend, uploads/, config.py / .env, LLM Service	10.6
Figure 10.5	Deployment	Recommended (minimal)	Documented runtime environment only	Client browser, Python (venv) runtime, uploads/, environment variable, LLM Service	10.7

None	State Machine	Not applicable	No object lifecycle or state transitions are specified	None	10.8 (explanation only)
------	------------------	-------------------	--	------	-------------------------------

Validation notes

- 1. Basis of the cross-check.** Every diagram was checked against the uploaded project PDF and the Project Knowledge Base. Chapters 3, 4, 5, 8 and 9 have not been generated yet, so the check against them is limited to the shared terminology. When those chapters are written they must use exactly these names: User Interface, File Upload, Document Extraction, Data Cleaning & Validation, Financial Analysis, Trend Analysis, LLM Explanation, Insight Generation, Dashboard, Report Generation, Flask Backend, LLM Service (external LLM/API).
- 2. How the images were produced.** PlantUML was not available in the environment used to prepare this file (no PlantUML installation and no network access). The images were therefore drawn with Graphviz and a Python drawing script in standard UML notation. The PlantUML code in Part B describes the same content but was **not executed here**. Render it with PlantUML before submission and check for syntax issues; layout may differ slightly from the images.
- 3. Deliberately excluded.** Authentication, administrator or other actors, databases (SQLite appears in the document only as an optional component with no module), APIs other than the LLM/API layer, cloud services, notification or payment systems.
- 4. Details the project document leaves open.** Whether analysis starts automatically after upload; how and in what format the report is delivered; how control passes between modules internally; the wording of error messages; the host machine for the Flask application; the files that implement File Upload and Data Cleaning & Validation. The chapter text states these as "Not specified in the project document."
- 5. Font.** This PDF uses Liberation Serif, which is metric-compatible with Times New Roman. Apply Times New Roman when the text is moved into Word.

CHAPTER 10

UML DIAGRAMS

10.1 Introduction to UML Representation

The LLM-Assisted Financial Statement Analysis Tool is a sequential pipeline: Upload, Extract, Clean, Validate, Calculate, Compare, Explain, Visualize and Report. It has one human user, ten modules (Chapter 9), an application layer built on Flask (Chapter 8) and one external service, the LLM Service. UML diagrams are used in this chapter to show the views in which this structure matters: what the user can do, the order of the processing steps and the decisions within them, how the modules hand data to one another, how the software is divided into components, and the environment in which it runs.

Only diagrams that the project document can support are included. Table 10.1 summarises the assessment of the seven standard diagram types. The class diagram and the state machine diagram are not included because the project document does not define the information they require; the reasons are given in Sections 10.5 and 10.8. All diagrams use the module and layer names of Chapters 8 and 9. Where the document does not specify a detail, the diagram omits it and the text states "Not specified in the project document."

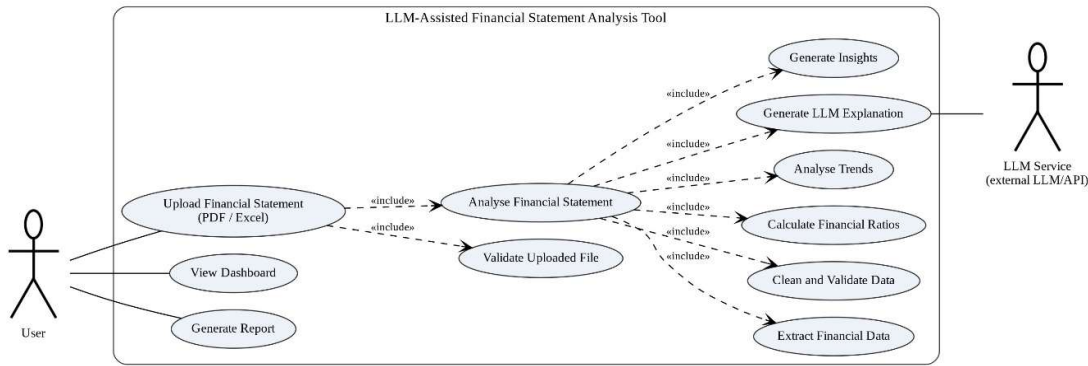


Figure 10.1: Use Case Diagram of the LLM-Assisted Financial Statement Analysis Tool

Figure 10.1 shows one primary actor, the User, associated with three use cases: Upload Financial Statement, View Dashboard and Generate Report. Upload Financial Statement includes Validate Uploaded File and Analyse Financial Statement, and the latter includes the six processing functions that correspond to the modules of Chapter 9. The include relation is used because the project workflow presents extraction, validation, calculation and explanation as the steps that follow an upload. Whether the analysis starts automatically after upload or through a separate user action is Not specified in the project document.

The LLM Service takes part only in Generate LLM Explanation. This reflects the design rule that the application performs the calculations and the LLM only explains verified results. No administrator, login or other actor is shown because none is specified.

10.3 Activity Diagram

Figure 10.2 represents the end-to-end workflow of one analysis, from the upload of a statement to the dashboard and the optional report. The diagram is drawn in two parts that are joined by connector A.

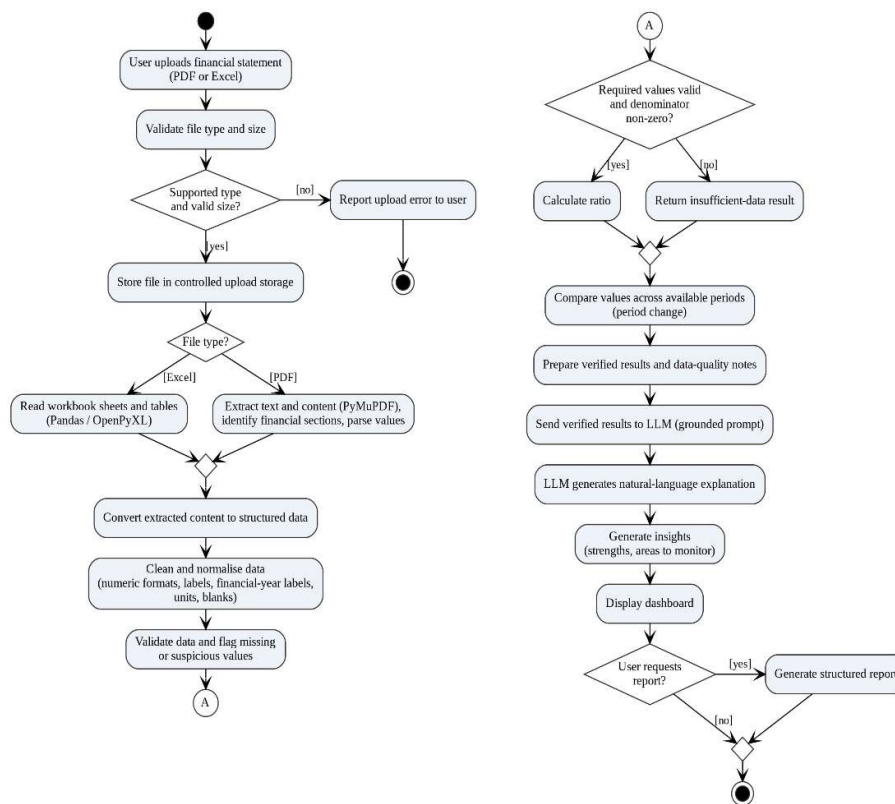


Figure 10.2: Activity Diagram of the Financial Statement Analysis Workflow

The workflow starts with file validation. A file of unsupported type or invalid size is rejected and an error is reported to the user, which corresponds to the error handling of the Application layer; the wording of the error is Not specified in the project document. An accepted file is stored and the activity branches by file type. Excel workbooks are read through Pandas and OpenPyXL. Text-based PDFs are processed through PyMuPDF, which includes identifying the relevant financial sections and parsing the values. Both branches merge into structured data. Scanned PDFs are not shown because OCR is future scope; their handling is Not specified in the project document.

The structured data is cleaned and normalised and then validated, and missing or suspicious values are flagged and not invented. The decision before the ratio calculation follows the stated rule: a ratio is calculated only when the required values are valid and the denominator is non-zero, otherwise an insufficient-data result is returned. The decision applies to each of the four indicators. The document states only that an unavailable result is returned, so the workflow is shown as continuing.

Period changes are then calculated, and the verified results and data-quality notes are sent to the LLM in a grounded prompt. Insight Generation follows the explanation, and the dashboard is displayed. Report generation is an action requested by the user. The document specifies trend analysis and areas to monitor but no separate risk analysis, so no risk analysis activity is shown.

10.4 Sequence Diagram

Figure 10.3 shows one complete analysis run for an accepted statement, followed by a report request. The lifelines are the User, the User Interface, the Flask Backend, the modules of Chapter 9 that take part, and the external LLM Service.

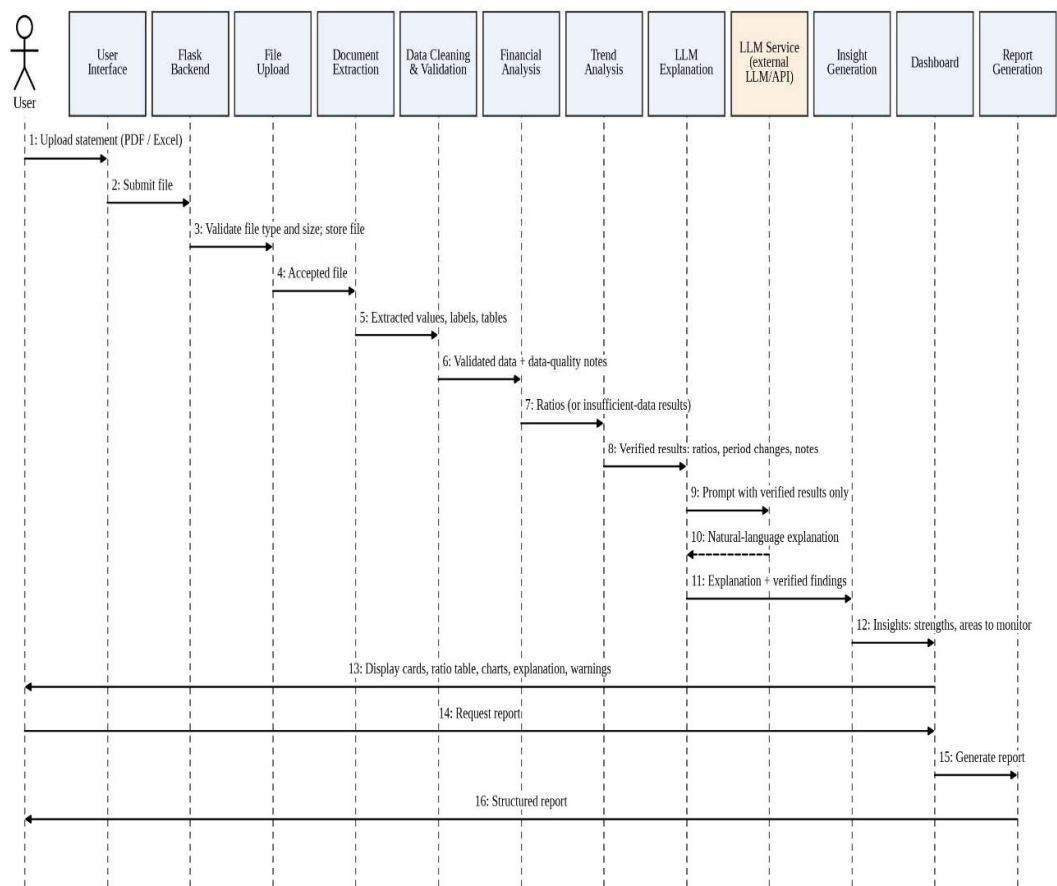


Figure 10.3: Sequence Diagram of an Analysis Run and Report Request

The messages follow the pipeline order of the system architecture. Each message names the data handed to the next stage, not a function name, because the interfaces and function names are Not specified in the project document. The LLM Explanation module sends a prompt that contains verified results only (message 9) and receives the natural-language explanation (message 10), which is the only interaction with the external LLM Service. The explanation and the verified findings then pass to Insight Generation, and the results are presented on the Dashboard

Component	Layer	Artifact in the recommended project structure
User Interface	Presentation	templates/index.html, templates/report.html, static/css/style.css
Dashboard	Presentation	templates/dashboard.html, static/js/dashboard.js; Chart.js
Flask Backend	Application	app.py, config.py
File Upload	Application	Not specified in the project document
Document Extraction	Extraction	extractor/excel_extractor.py, extractor/pdf_extractor.py
Data Cleaning & Validation	Data Preparation	Not specified in the project document
Financial Analysis	Analysis	analysis/ratios.py
Trend Analysis	Analysis	analysis/trends.py
LLM Explanation	AI/LLM	llm/explainer.py
Insight Generation	Insights	analysis/insights.py

The document assigns workflow control to the Flask Backend, but how control is transferred between the modules internally is Not specified in the project document, so the diagram shows only the hand-offs. If the file is rejected at message 3, an error is reported to the user and the sequence ends. Messages 14 to 16 show the report generation action. The delivery format of the report is Not specified in the project document.

10.5 Class Diagram

A class diagram is not included in this report. The project document identifies the modules, the layers and a recommended file structure, but it does not define classes, attributes, operations, associations or inheritance. Class names, fields and methods would therefore have to be invented, which is not permitted for this report. These details are Not specified in the project document. The static structure of the software is represented at module level by the component diagram in Section 10.6.

10.6 Component Diagram

Figure 10.4 shows the software components and their dependencies. The components are the modules of Chapter 9 and the Flask Backend of the Application layer. The artifacts associated with each component are taken from the recommended project structure and are listed in Table 10.4.

Table 10.4: Components and Their Documented Artifacts

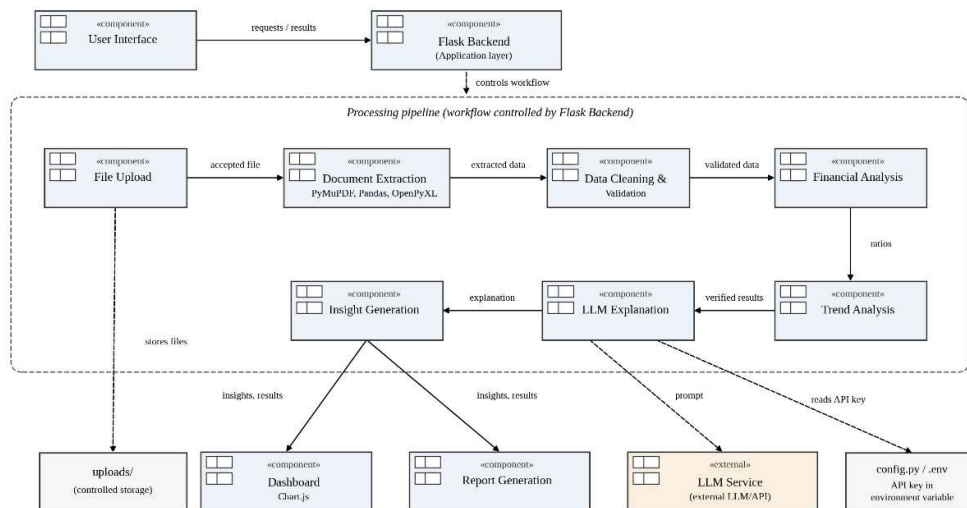
The recommended structure in the project document is a plan, and it states that modules are to be added as the roadmap progresses. The file names are therefore planned artifacts and not confirmed implementation. The assignment of a file to a component is made by name and is not stated explicitly in the project document.

Figure 10.4: Component Diagram of the LLM-Assisted Financial Statement Analysis Tool

In Figure 10.4 the User Interface sends requests to the Flask Backend, which controls the workflow of the processing pipeline. File Upload stores accepted files in uploads/ and

passes them to Document Extraction, which depends on PyMuPDF, Pandas and OpenPyXL. The data then moves through Data Cleaning & Validation, Financial Analysis and Trend Analysis. LLM Explanation depends on the external LLM Service and reads the API key from an environment variable, in line with the rule that secrets are not stored in code. The Dashboard and Report Generation use the final results of the pipeline.

The project document defines no interfaces or ports, so the components are connected by the data they exchange. The option



al

SQLite database is not shown because the document marks it as optional and assigns it no module

10.7 Deployment Diagram

The project document gives limited deployment information, so Figure 10.5 shows only the documented elements: the browser interface, the Flask application running in a Python

virtual environment, the upload storage, the API key held in an environment variable, and the external LLM Service.

Figure 10.5: Deployment Diagram Showing the Documented Runtime Environment

